ZipClip: Edit Less, Post More

A PROJECT REPORT (PHASE-2)

by

JESVIN J. (VJC22CSD032)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND DESIGN

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

APRIL 2026

ZipClip: Edit Less, Post More

A PROJECT REPORT (PHASE-2)

by

JESVIN J. (VJC22CSD032)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND DESIGN

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

under the guidance

of

Mrs. Sabitha Raju

Head of Department, CG

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN
VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

APRIL 2026

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

Department of Computer Science and Design

Vision

Moulding socially dynamite Computer Science and Design engineers capable of driving technological advancements.

Mission

1. To provide comprehensive education that combines technical knowledge, creativity and critical thinking through computer science principles and design methodologies.
2. Promote research and industry collaboration to empower our graduates for addressing real world challenges.
3. To cultivate ethical values, social responsibility and a commitment to lifelong learning among students.

Programme Educational Objectives

Our Graduates

1. Shall have strong foundation in theoretical and practical aspects of Computer Science and Design principles to develop innovative solutions.
2. Shall possess effective communication, teamwork and leadership skills to collaborate with diverse stakeholders and to practice in a corporate firm.
3. Shall have potential to become entrepreneurs, innovators and pursue research in Computer Science and Design or related fields.
4. Shall have a strong sense of social awareness and stay updated with evolving design technologies.

Program Outcomes

1. **Engineering knowledge**:Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**:Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences
3. **Design / development of solutions**:Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**:Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**:Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:**Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:**Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:**Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice
9. **Individual and team work:**Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings
10. **Communication:**Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and unread in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning**:Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes

1. Ability to integrate theory and practice to design software applications.
2. Able to apply computer science skills and design tools to analyse, design and model composite systems.
3. Ability to design and manage projects to develop a career in a related industry.

VISWAJYOTHI COLLEGE OF ENGINEERING AND TECHNOLOGY, VAZHAKULAM

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

BONAFIDE CERTIFICATE

This is to certify that the project report (phase-2) entitled " **ZipClip: Edit Less, Post More** " is a bonafide record of the work done by **JESVIN J (VJC22CSD032)** in partial fulfillment of the requirements for the award of the **Degree of Bachelor of Technology** in **Computer Science and Design** of APJ Abdul Kalam Technological University.

Internal Supervisor **External Supervisor**

Project Coordinator **Head of Department**

ACKNOWLEDGEMENT

First and foremost, I thank **God Almighty** for His divine grace and blessings in making all these possible. May he continue to lead us in the years. It is my privilege to render my heartfelt thanks to our beloved Manager, **Rev. Msgr. Dr. Pius Malekandathil** , our director **Rev. Fr. Paul Parathazham** and our Principal **Dr. K K Rajan** for providing me the opportunity to do this project report (phase-2) during the Fourth year (2026) of my B.Tech degree course. I am deeply thankful to our Head of the Department and our Project Guide, **Mrs. Sabitha Raju** for her support and encouragement. I also express sincere thanks to our Project Coordinator **Mrs. Anju Markose**, Assistant Professor, Department of Computer Science and Design for her guidance and support. I also thank all the staff members of the Computer Science and Design Department for providing their assistance and support. Last, but not the least, I Would like to thank all my friends and family for their valuable feedback from time to time as well as their help and encouragement.

DECLARATION

I undersigned hereby declare that the project report "**ZipClip: Edit Less, Post More**", submitted for partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of the APJ Abdul Kalam Technological University is a bonafide work done by myself under the supervision of **Mrs. Sabitha Raju**. This submission represents ideas in our own words and where ideas or words of others have been included, We have adequately and accurately cited and referenced the original sources. I also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or formed the basis for the award of any degree, diploma or similar title of any other University.

Place: Vazhakulam

Date: 01-04-2026

JESVIN J

ABSTRACT

ZipClip is an innovative AI-powered application developed to revolutionize the process of short-form video creation from lengthy video content. In today's digital landscape, short-form videos dominate social media platforms such as YouTube Shorts, Instagram Reels, and TikTok, making rapid content generation a vital need for creators. However, traditional manual editing is not only time-intensive but also requires creative and technical expertise that many users may lack. ZipClip overcomes these limitations by automating the highlight generation process through advanced artificial intelligence and machine learning techniques.The system employs a multi-model AI architecture that integrates scene detection, speech and silence analysis, emotion recognition, and content scoring mechanisms to identify the most engaging portions of a video. Each module contributes to the overall understanding of the video's narrative — for instance, emotion recognition detects expressive moments, while content scoring evaluates their relevance and viewer appeal. This holistic approach ensures that the extracted clips maintain the storytelling intent of the original video while enhancing engagement value. In addition to video-based highlights, ZipClip also supports photo-based storytelling, where users can upload personal or professional photos to generate dynamic story reels enhanced with AI-curated transitions, animations, and background music. This functionality extends the application's usability beyond traditional video editors, catering to social media influencers, educators, marketing professionals, and casual users alike. The output is automatically optimized for multiple platforms, ensuring correct aspect ratios, durations, and quality settings based on the destination channel. Furthermore, ZipClip features an interactive dashboard where users can preview generated content, make fine-tuned adjustments, and export final results effortlessly. By integrating automation with creative flexibility, ZipClip drastically reduces editing time, improves consistency, and enables users to focus more on storytelling rather than technical post-production tasks. Ultimately, it transforms long-form media into concise, visually appealing narratives that align with modern digital consumption trends.

Contents

List of Figures

List of Abbreviations

AI	Artificial Intelligence
ASR	Automatic Speech Recognition
CLIP	Contrastive Language-Image Pretraining
CNN	Convolutional Neural Network
DCNN	Deep Convolutional Neural Network
LSTM	Long Short-Term Memory
ML	Machine Learning
MLLM	Multimodal Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition

Chapter 1

INTRODUCTION

In today's fast-paced digital landscape, short-form video content has become one of the most dominant forms of media consumption. Platforms such as YouTube Shorts, Instagram Reels, and TikTok have revolutionized how users create and engage with videos, emphasizing brevity, creativity, and instant impact. As a result, creators, educators, and businesses are constantly seeking efficient methods to repurpose long-form videos into concise, captivating snippets that attract more viewers and maintain engagement. However, the process of manual video editing remains time-consuming, inconsistent, and skill-dependent, often leading to creative fatigue and reduced productivity.

ZipClip aims to solve these challenges by introducing an AI-powered automation platform that intelligently identifies and extracts the most meaningful, engaging, and visually appealing moments from lengthy video footage. By integrating artificial intelligence (AI), machine learning (ML), and natural language processing (NLP) technologies, the system performs a deep analysis of the video's visual scenes, emotions, and narrative flow. The AI evaluates content based on multiple parameters, such as expression intensity, speech dynamics, scene transitions, and viewer engagement potential, ensuring that each extracted highlight retains context and storytelling coherence.

ZipClip is developed as a web-based application designed to be intuitive and user-friendly. It leverages multi-model AI architecture that includes modules for scene detection, speech and silence analysis, emotion recognition, and content scoring.The platform also supports photo-based storytelling, allowing users to transform their personal or professional images into visually rich story reels, enhanced by AI-curated effects, transitions, and background music.

In essence, ZipClip redefines modern video editing by combining the intelligence of AI with the flexibility of user-driven customization. It enables users to "edit less and post more", ensuring that high-quality, engaging short-form content can be generated effortlessly, thereby meeting the evolving demands of today's digital content ecosystem.

1.1 Problem Statement

Long-form videos often hide the most engaging moments, but finding them manually is slow and exhausting. Many creators lack the time, skills, or resources to transform raw footage into short, polished clips. Existing editing tools are either too complex or too expensive for everyday users. Missing the right highlights means lost audience attention and reduced impact on short-form platforms.

1.2 Objective

Automate the process of extracting highlights from long-form videos using AI-based multi-modal analysis. Provide an interactive platform for creators to preview, refine, and export clips optimized for YouTube Shorts, Instagram Reels, and TikTok. Enable easy content creation by allowing users to generate story reels from their own photos with AI-driven transitions, captions, and music. Ensure clips preserve narrative flow and emotional impact, making short-form content more engaging and platform-ready. Support platform-specific formatting so clips are instantly ready for YouTube Shorts, Instagram Reels, and TikTok. Incorporate natural language prompts (e.g., “show all funny parts”) for personalized highlight generation. Ensure scalability with a modular AI design that allows adding new agents for education, sports, or event videos. Provide a foundation for future enhancements like auto-music selection, subtitles, and multi-language support.

1.3 Scope

AI-Powered Video Summarization: Automatically analyze long-form videos to detect scenes, emotions, and key moments, producing short-form highlights. Interactive Content Creation: Provide an intuitive dashboard where users can preview, fine-tune, and export clips optimized for YouTube Shorts, Instagram Reels, and TikTok. Story Reel Generation: Allow users to upload personal photos and create AI-driven reels with captions, transitions, and music. Scalable and Extensible Design: Support modular AI agents, making the system adaptable for future use cases like sports highlights, educational content, and event coverage.

Chapter 2

LITERATURE SURVEY

2.1 SumBot: Summarize Videos Like a Human

The research paper titled “SumBot: Summarize Videos Like a Human” introduces an intelligent video summarization framework that aims to replicate the way humans create concise and meaningful summaries from long videos. Traditional video summarization techniques often focus on visual similarity or keyframe extraction, which can result in summaries that lack narrative flow or emotional context. In contrast, SumBot is designed to mimic the cognitive process of a human editor, selecting the most relevant and contextually rich shots that effectively convey the story or essence of the original video[1].

The system integrates several stages in its workflow. First, shot splitting is performed to divide the video into smaller, manageable segments known as shots. This step ensures that the system can analyze each part independently and identify important transitions or visual changes. Next, feature extraction is carried out using deep learning models such as Inception and Convolutional Neural Networks (CNNs). These models capture both low-level visual details (like color and texture) and high-level semantic features (such as objects, scenes, and actions). The extracted features provide a strong representation of the video’s content, enabling the system to understand its structure and meaning.

2.1.1 Methodology of SumBot: Summarize Videos Like a Human

The methodology involves shot splitting to divide videos into segments, extracting features using Inception/CNN models, and applying Markov Decision Process (MDP) and Inverse Reinforcement Learning (IRL) with Maximum Entropy (MaxEnt) reward functions and policy iteration to select optimal summaries that mimic expert human editing.

2.1.2 Model Architecture

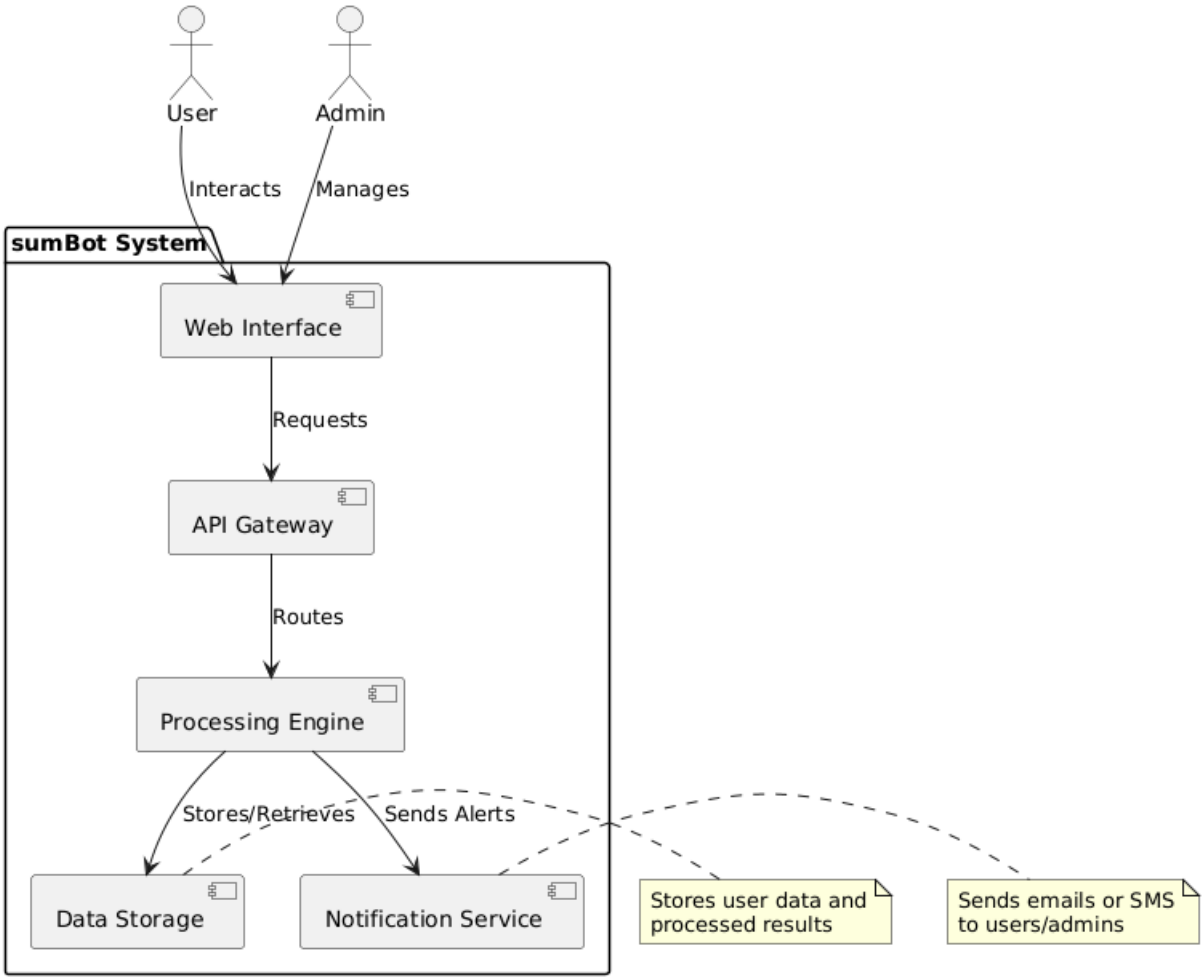

Figure 2.1: SumBot architecture

2.1.3 Advantages

1. Mimics expert human summarization with high F-score.
2. Effective feature extraction using Inception/CNN.
3. Uses MDP/IRL for optimal policy learning.

2.1.4 Disadvantages

1. Linear reward function limits complexity.
2. Focused on semi-structured videos only.
3. Soccer-focused; limited generalizability.

2.2 Video summarization using deep learning techniques: a comprehensive review

The paper titled “Video Summarization Using Deep Learning Techniques: A Comprehensive Review” provides an in-depth analysis of various deep learning approaches developed for generating concise, meaningful, and informative summaries of long video content. With the explosive growth of video data on platforms such as YouTube, Instagram, and educational archives, efficient video summarization has become a critical area of research. The review highlights how deep learning techniques have revolutionized this domain by enabling systems to automatically extract and present the most important segments of a video while maintaining temporal coherence and narrative quality[2].

The review begins by discussing the fundamental goal of video summarization—reducing the duration of a video without losing its essential information or storytelling elements. Traditional summarization methods relied on low-level visual features like color histograms and motion vectors. However, such techniques often failed to capture the semantic meaning of scenes. With the emergence of deep learning, models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory networks (LSTMs) have enabled a shift toward more context-aware and intelligent summarization systems.

2.2.1 Methodology

The methodology adopted in “Video Summarization Using Deep Learning Techniques: A Comprehensive Review” is primarily based on an extensive survey and comparative analysis of existing deep learning frameworks developed for video summarization. The authors aim to provide a detailed understanding of how different neural network architectures—especially Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) models—have contributed to the evolution of this field. The review systematically explores both supervised and unsupervised approaches, evaluating their effectiveness, scalability, and suitability for various types of video data and application domains.

2.2.2 Model Architecture

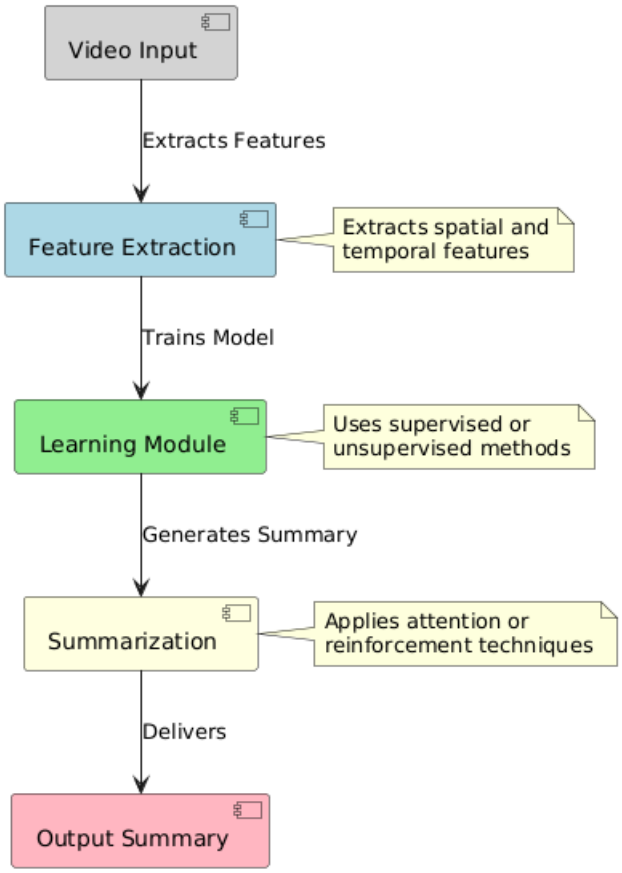

Figure 2.2: Video Summarization Review architecture

2.2.3 Advantages

1. Covers both supervised and unsupervised methods.
2. Scalable for platforms.
3. Provides a comprehensive comparison of multiple deep learning architectures like CNN, LSTM, and hybrid models.
4. Highlights recent advancements and trends, helping researchers identify future directions.

2.2.4 Disadvantages

1. Limited real-time evaluation.
2. Bias in datasets.
3. Lack of standardized benchmark datasets, making fair comparison difficult.
4. Difficulty in interpreting model decisions, reducing transparency in summarization outcomes.

2.3 Video summarization for event-centric videos

The paper titled “Video Summarization for Event-Centric Videos” presents a specialized approach to summarizing videos that revolve around specific events, such as sports matches, concerts, conferences, or social gatherings. Unlike general-purpose summarization techniques that treat all video content uniformly, this method focuses on identifying and extracting event-driven highlights that capture the key moments, actions, and emotions associated with a particular event. The goal is to generate concise summaries that not only shorten the viewing time but also retain the contextual and emotional essence of the original video[3].

The system integrates a multimodal feature fusion strategy, combining visual, audio, and sometimes textual information (such as subtitles or metadata) to achieve a richer understanding of the video content. Visual features, such as player movements or crowd reactions, are extracted using Convolutional Neural Networks (CNNs), while audio features like cheering, applause, or commentary are analyzed to detect moments of excitement or importance. This multimodal fusion ensures that the summarization process goes beyond mere visual cues and considers the broader sensory and contextual information present in the video.

2.3.1 Methodology

he methodology proposed in “Video Summarization for Event-Centric Videos” centers around an event-driven framework that emphasizes the fusion of multimodal features to extract meaningful highlights while preserving the contextual flow of the video. Unlike traditional summarization methods that rely solely on visual information, this approach integrates multiple data sources—such as visual, audio, and textual cues—to achieve a deeper understanding of the events occurring within the video.

The process begins with event detection and segmentation, where the video is divided into smaller temporal segments or shots based on significant changes in visual or audio patterns. Each segment is analyzed to determine its relevance to the main event. Convolutional Neural Networks (CNNs) are used to extract spatial and semantic features from the visual stream, capturing key elements like people, objects, or activities. At the same time, audio feature extraction techniques identify crucial auditory signals such as applause, cheering, or tone changes in speech, which often indicate moments of importance or excitement.

2.3.2 Model Architecture

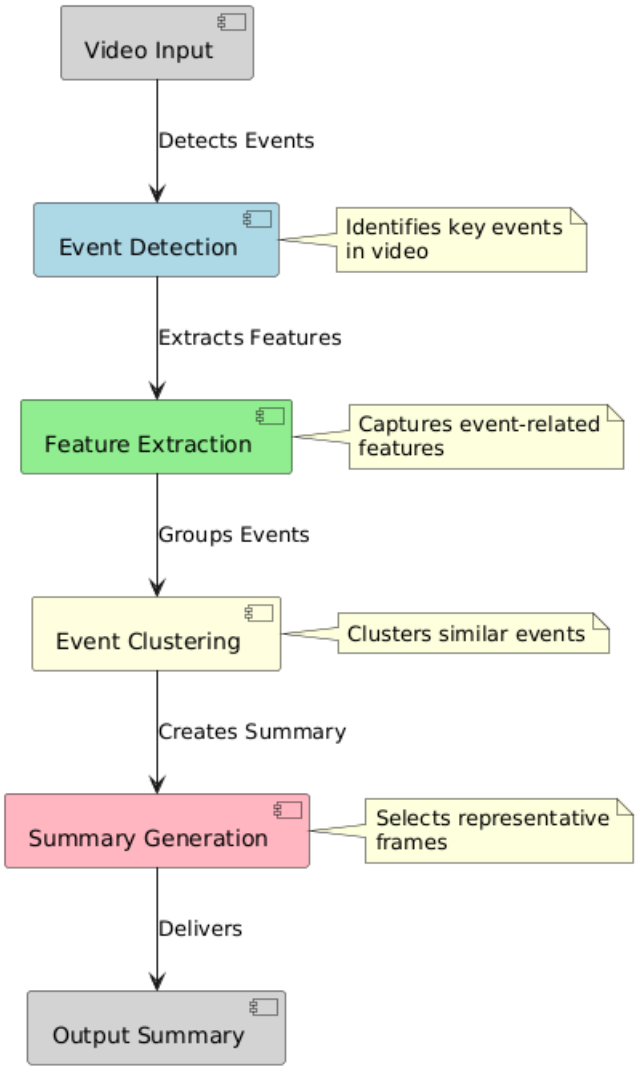

Figure 2.3: Event-Centric Video Summarization architecture

2.3.3 Advantages

1. Effective for event highlights.
2. Multimodal fusion enhances context.

2.3.4 Disadvantages

1. Not tested on very short-form content.
2. Limited domain adaptation.

2.4 AI-driven video summarization for optimizing content search and retrieval efficiency

TThe paper titled “AI-Driven Video Summarization for Optimizing Content Search and Retrieval Efficiency” explores the use of artificial intelligence techniques to enhance how videos are summarized, indexed, and retrieved across large-scale content platforms. With the massive increase in online video data, efficient search and retrieval have become crucial for both end-users and content creators. This research focuses on developing intelligent summarization systems that can automatically extract spatial and temporal highlights, thereby improving accessibility, searchability, and automation in content management workflows[4].

The primary objective of the system is to create summaries that not only condense video length but also serve as metadata-rich representations suitable for search engines and recommendation systems. By analyzing both spatial features (objects, faces, and scenes) and temporal dynamics (events, transitions, and motion patterns), the proposed AI-driven framework ensures that the summaries maintain semantic coherence and relevance. This allows users to quickly preview, locate, and retrieve specific moments within lengthy video archives.

2.4.1 Methodology

The methodology presented in “AI-Driven Video Summarization for Optimizing Content Search and Retrieval Efficiency” focuses on combining spatial and temporal analysis to produce intelligent video summaries that enhance both content accessibility and retrieval performance. This approach leverages deep learning models to extract, interpret, and organize meaningful video features, enabling faster and more accurate search operations across large video repositories.

The process begins with spatial feature extraction, where each video frame is analyzed using Convolutional Neural Networks (CNNs). These networks identify and encode visual elements such as objects, faces, actions, and background scenes. By capturing these spatial cues, the system generates high-level semantic representations that describe the visual content of each segment. This step ensures that key frames are not only visually distinct but also semantically informative.

2.4.2 Model Architecture

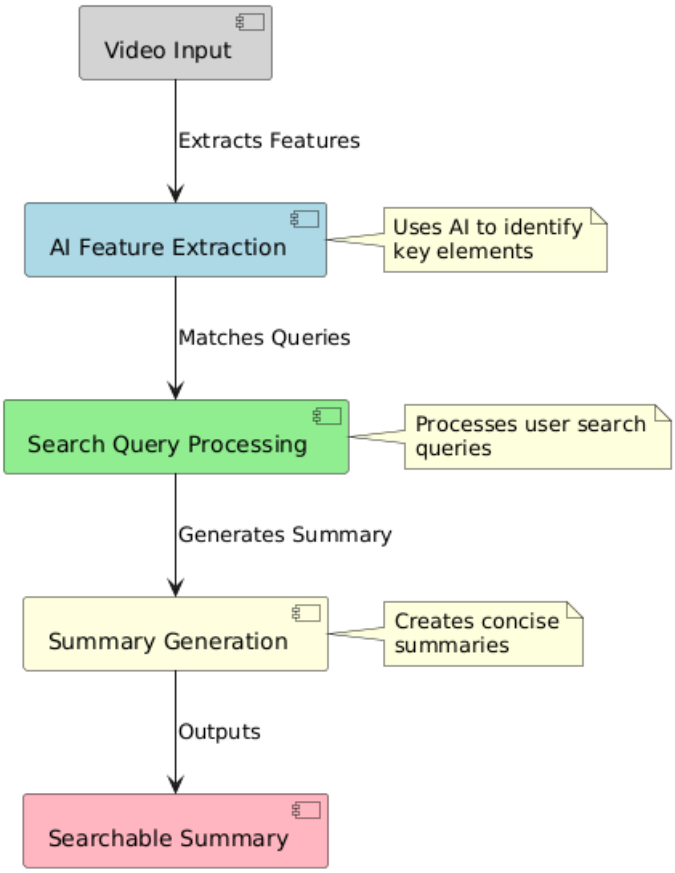

Figure 2.4: AI-Driven Video Summarization for Search architecture

2.4.3 Advantages

1. Optimized for retrieval.
2. Automation for creators.
3. Combines spatial and temporal analysis for more context-aware summarization.
4. Enhances search accuracy by indexing videos with semantic tags and feature embeddings.

2.4.4 Disadvantages

1. Focus on search, not platform deployment.
2. Evaluation on YouTube only.
3. Limited real-time performance when handling long or high-resolution videos.
4. Lacks user interaction or feedback mechanisms to refine summary quality.

2.5 Video summarization via knowledge-aware multimodal networks

The paper titled “Video Summarization via Knowledge-Aware Multimodal Networks” proposes a novel approach to video summarization that integrates domain knowledge with multimodal feature fusion to produce more robust and contextually relevant highlights. Unlike conventional methods that rely solely on visual or temporal patterns, this framework incorporates prior knowledge about the content, events, or domain to guide the summarization process, ensuring that the selected highlights are meaningful and aligned with human understanding[5].

A key innovation of this work is the use of knowledge-driven multimodal fusion. The system combines inputs from multiple modalities, including visual cues (frames and objects), audio signals (speech, music, or environmental sounds), and textual metadata (subtitles or captions). Additionally, it leverages knowledge graphs or domain-specific ontologies to provide context about relationships between objects, events, or actions. This allows the model to prioritize segments that are semantically significant rather than merely visually salient.

2.5.1 Methodology

The methodology of “Video Summarization via Knowledge-Aware Multimodal Networks” revolves around leveraging knowledge-driven deep learning frameworks to fuse multimodal video data and extract meaningful highlights. The approach integrates information from multiple sources, including visual frames, audio signals, and textual metadata, while incorporating domain knowledge to guide the summarization process and improve semantic relevance.

The process begins with multimodal feature extraction, where each modality is processed by a dedicated agent. Visual features are captured using Convolutional Neural Networks (CNNs) to identify objects, scenes, and key visual events. Audio signals, such as speech, music, or environmental sounds, are analyzed using spectrogram-based neural networks to detect moments of interest. Textual information, including subtitles or captions, is processed using natural language processing techniques to extract semantic context and relevant keywords.

2.5.2 Model Architecture

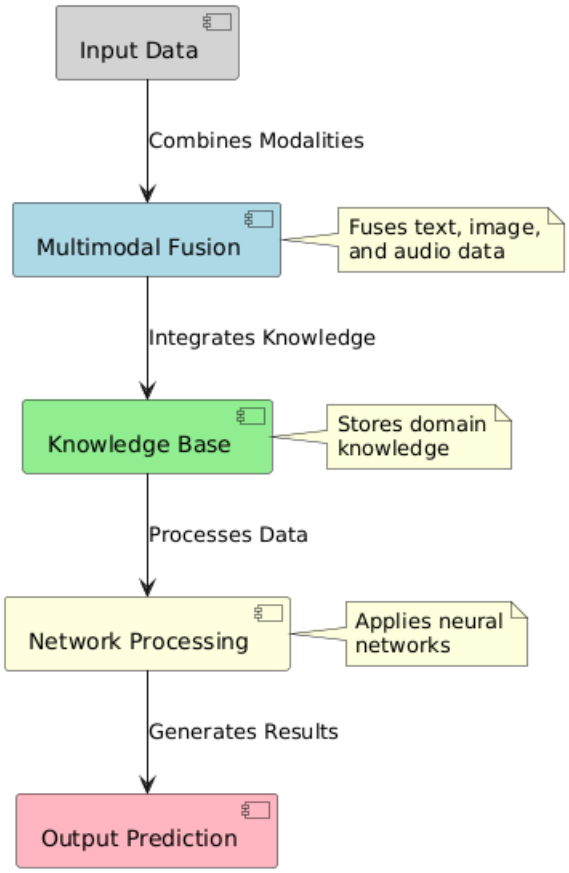

Figure 2.5: Knowledge-Aware Multimodal Networks architecture

2.5.3 Advantages

1. Robust highlight extraction.
2. Knowledge-driven fusion.
3. Maintains temporal and semantic coherence in the generated summaries.
4. Multi-agent collaboration enables better integration of visual, audio, and textual features.

2.5.4 Disadvantages

1. Computational cost.
2. Sparse real-user studies.
3. Dependency on accurate knowledge graphs or domain-specific ontologies, which may not always be available.
4. Potential delays in real-time summarization for long or high-resolution videos.

2.6 Zero-shot scene change detection

The paper titled “Zero-Shot Scene Change Detection” introduces a novel approach for detecting scene transitions in videos without requiring any prior training or labeled datasets. Traditional scene change detection methods often rely on supervised learning models, which need large annotated datasets and extensive training to recognize shot boundaries accurately. In contrast, this study presents a zero-shot framework, enabling the system to identify scene changes directly from the video content itself, making it highly adaptable to new or unseen datasets and reducing reliance on manual annotations[6].

The proposed method leverages feature differencing techniques to detect boundaries between consecutive frames or shots. Visual features are extracted from frames using pre-trained deep learning models, such as CNNs, which encode spatial information including objects, textures, and scene layouts. By computing differences between consecutive frame features, the system identifies abrupt or gradual changes that signify scene transitions. This allows for effective detection of both hard cuts and soft transitions, such as fades or dissolves, which are common in edited videos.

2.6.1 Methodology

The methodology of “Zero-Shot Scene Change Detection” focuses on identifying scene boundaries in videos without requiring any prior training or labeled datasets, making it highly adaptable to a wide variety of video content. The approach relies primarily on feature differencing, a technique that quantifies changes between consecutive frames or shots to detect significant transitions.

The process begins with frame-level feature extraction, where each frame is encoded using pre-trained deep learning models such as Convolutional Neural Networks (CNNs). These models capture rich visual information, including objects, textures, colors, and spatial layouts, providing a high-level representation of each frame. By comparing the features of consecutive frames, the system calculates the magnitude of change, which serves as an indicator of potential scene boundaries.

To improve robustness, the methodology often incorporates temporal smoothing techniques, which help to filter out minor fluctuations caused by camera movements, lighting changes, or background variations. This ensures that only significant changes—indicative of actual scene transitions—are detected. Additionally, adaptive thresholding is applied to differentiate between hard cuts, soft transitions, and gradual scene changes, allowing the system to handle a variety of editing styles commonly found in movies, TV shows, and online videos.

2.6.2 Model Architecture

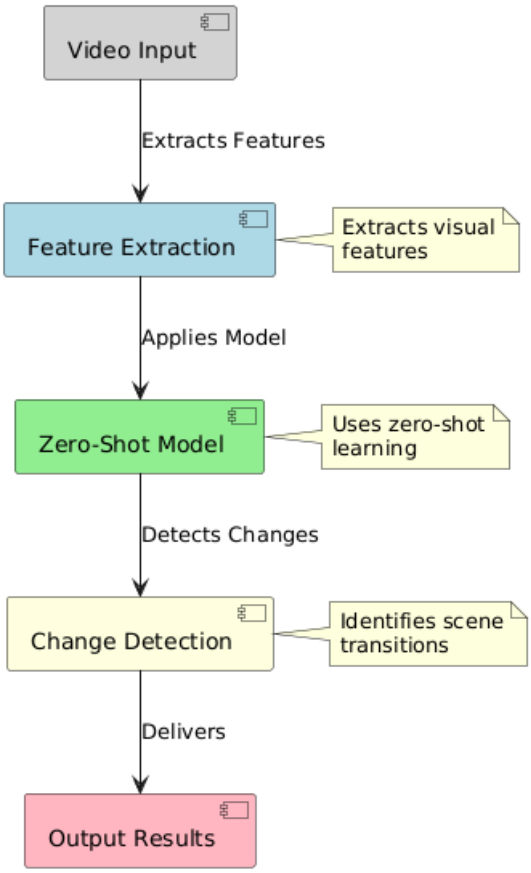

Figure 2.6: Zero-Shot Scene Change Detection architecture

2.6.3 Advantages

1. No training required.
2. Effective for boundary detection.
3. Scalable for large datasets without the need for labeled data.
4. Robust to different video genres and editing styles.

2.6.4 Disadvantages

1. May miss subtle transitions.
2. Limited evaluation on user videos.
3. Potential sensitivity to minor camera motion or lighting changes if temporal smoothing is insufficient.
4. Lacks integration of audio or textual cues, which could improve boundary detection in complex scenes.

Chapter 3

PROPOSED SYSTEM

The proposed system **ZipClip: Edit Less, Post More** aims to simplify and automate the process of converting long-form video content into engaging short-form clips suitable for modern social media platforms. The system leverages advanced artificial intelligence technologies, including speech recognition, natural language processing, and automated video processing techniques, to identify and extract the most meaningful segments from longer videos.

At its core, the system analyzes video content by first extracting the audio and generating a transcript using speech recognition technology. This transcript is then processed using AI-based language models to identify the most engaging or informative portions of the video. Based on this analysis, the system automatically selects highlight segments and converts them into short video clips optimized for platforms such as YouTube Shorts, Instagram Reels, and other mobile-first media platforms.

Beyond the automated highlight detection, the system provides several additional functionalities to improve the quality and usability of the generated clips. These include intelligent video cropping to adapt videos into vertical 16:9 format, automatic subtitle generation for improved accessibility, and seamless integration with a user-friendly web interface that allows users to upload videos and manage the processing workflow. The system also supports multiple video input methods, such as YouTube URLs and locally uploaded files, making it flexible and convenient for users.

By combining artificial intelligence techniques with powerful multimedia processing tools, the ZipClip system provides an efficient solution for automated video editing and highlight generation. This approach significantly reduces the time and effort required to manually edit long videos, enabling content creators, educators, and marketers to quickly produce high-quality short-form videos that are ready to be shared across various digital platforms.

3.1 Process Overview

Project Planning

a. Define Project Scope and Objectives: Clearly outline the goal of ZipClip, which is to automate the process of converting long videos into short clips using AI.

b. Identify Target Audience: Understand the needs of content creators, educators, and marketers who require quick and efficient video editing for short-form platforms.

c. Establish Project Timeline and Milestones: Create a development timeline with milestones to track the progress of system design, development, testing, and deployment.

d. Assemble Project Team: Form a team with expertise in AI development, web development, and system design to build the platform effectively.

Requirements Gathering and Analysis

a. Gather User Requirements: Identify user needs such as efficient video editing, automated clip detection, and customization options.

b. Analyze User Feedback: Analyze user feedback to identify common needs and prioritize important system features.

c. Define Functional and Non-functional Requirements: Define functional requirements such as AI-based video analysis and clip generation, and non-functional requirements such as system performance, usability, and reliability.

Design and Development.

a. App Design: Design an intuitive dashboard interface that allows users to upload videos and manage generated clips easily.

b. AI Integration: Develop AI modules for video content detection and clip scoring to identify important segments.

c. Data Processing: Implement video processing mechanisms to analyze uploaded videos and extract relevant segments.

d. Software Development: Develop the system using React.js for the front-end, Node.js for the back-end, and Python for AI processing.

Testing and Quality Assurance

a. Unit Testing: Test individual modules of the system to ensure each component functions correctly.

b. Integration Testing: Verify that all system components work together smoothly during the video processing workflow.

c. User Acceptance Testing (UAT): Allow users to test the system to evaluate usability and functionality.

d. Performance Testing: Test the system under different workloads to ensure stable performance.

Deployment and Maintenance

a. Deployment: Deploy the system as a web application accessible through a browser.

b. Monitoring and Support: Monitor system performance and provide support based on user feedback.

c. Updates and Feature Enhancements: Release updates to improve security, performance, and introduce new features.

3.1.1 Process Flow Diagram

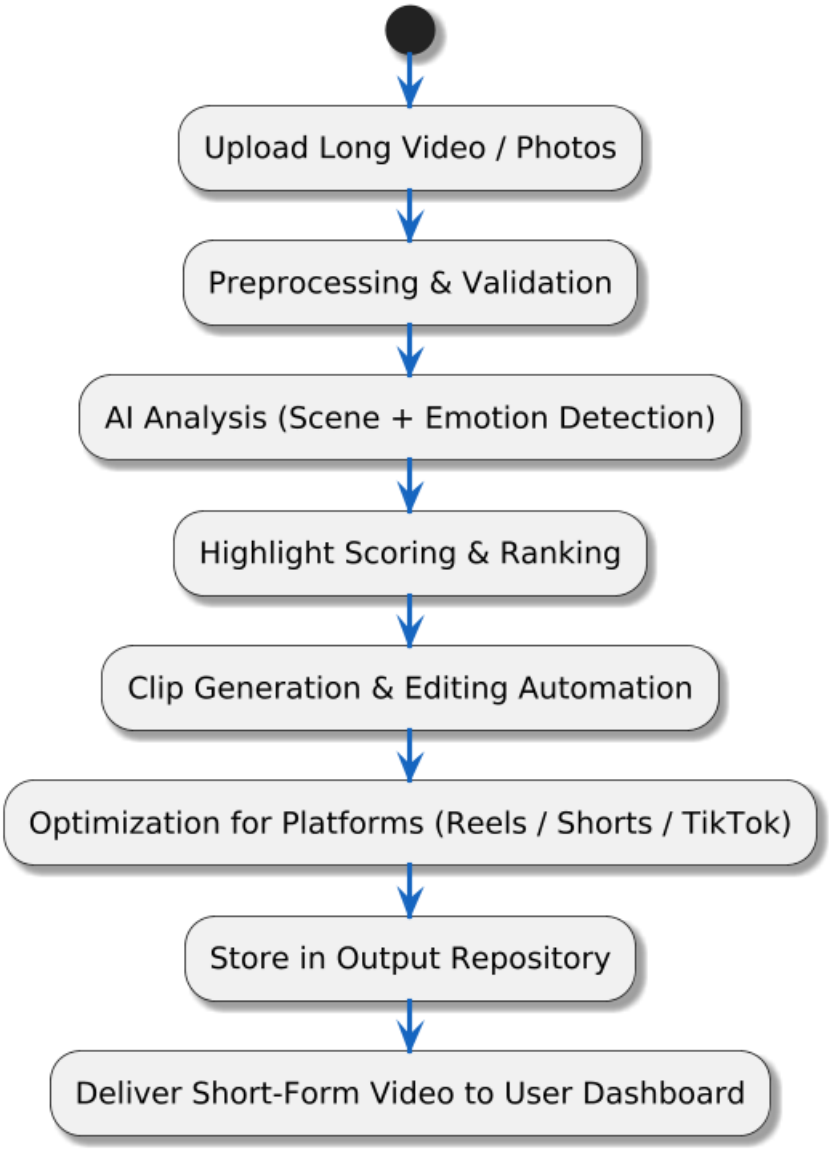

Figure 3.1: Process Flow Diagram

3.1.2 Architecture Diagram

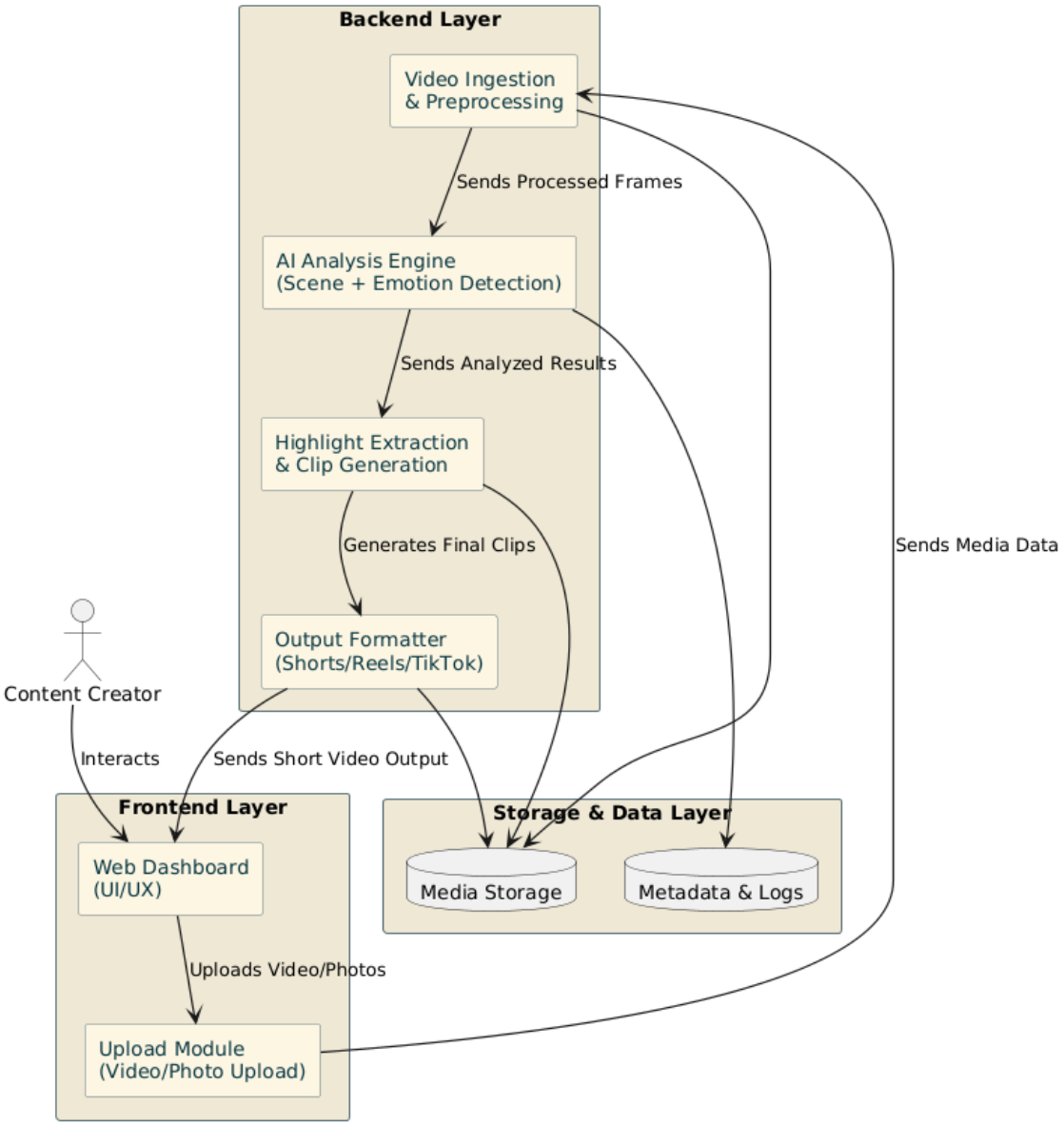

Figure 3.2: Architecture Diagram

3.1.3 Use case Diagram

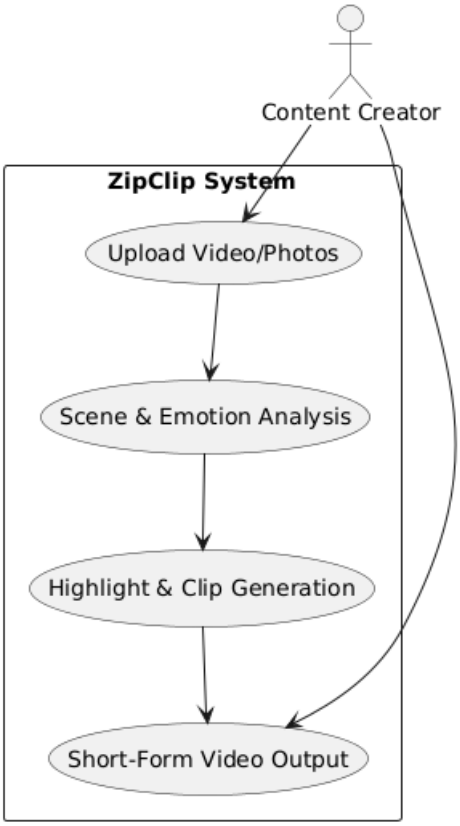

Figure 3.3: Use case diagram

3.1.4 Data flow diagram

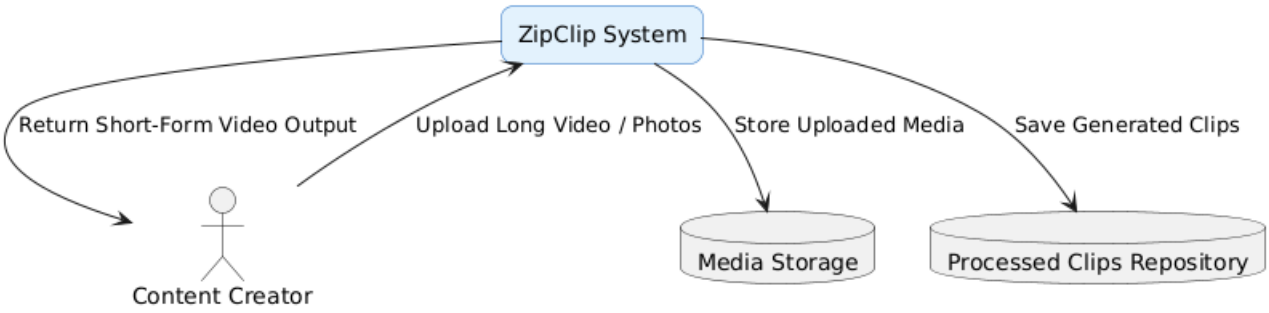

Figure 3.4: Data Flow Diagram Level 0

3.1.5 Data flow diagram

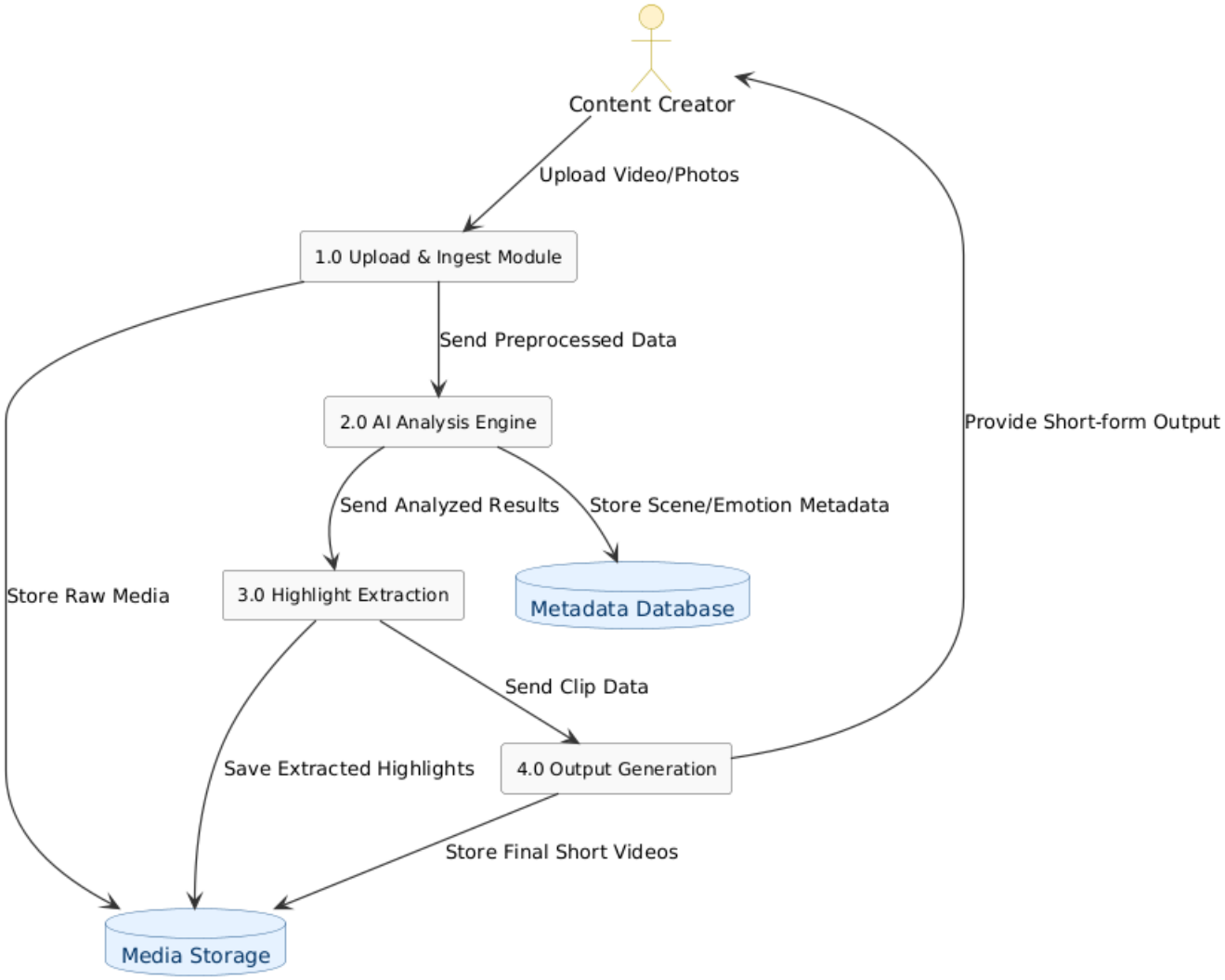

Figure 3.5: Data Flow Diagram Level 1-User

3.1.6 Class Diagram

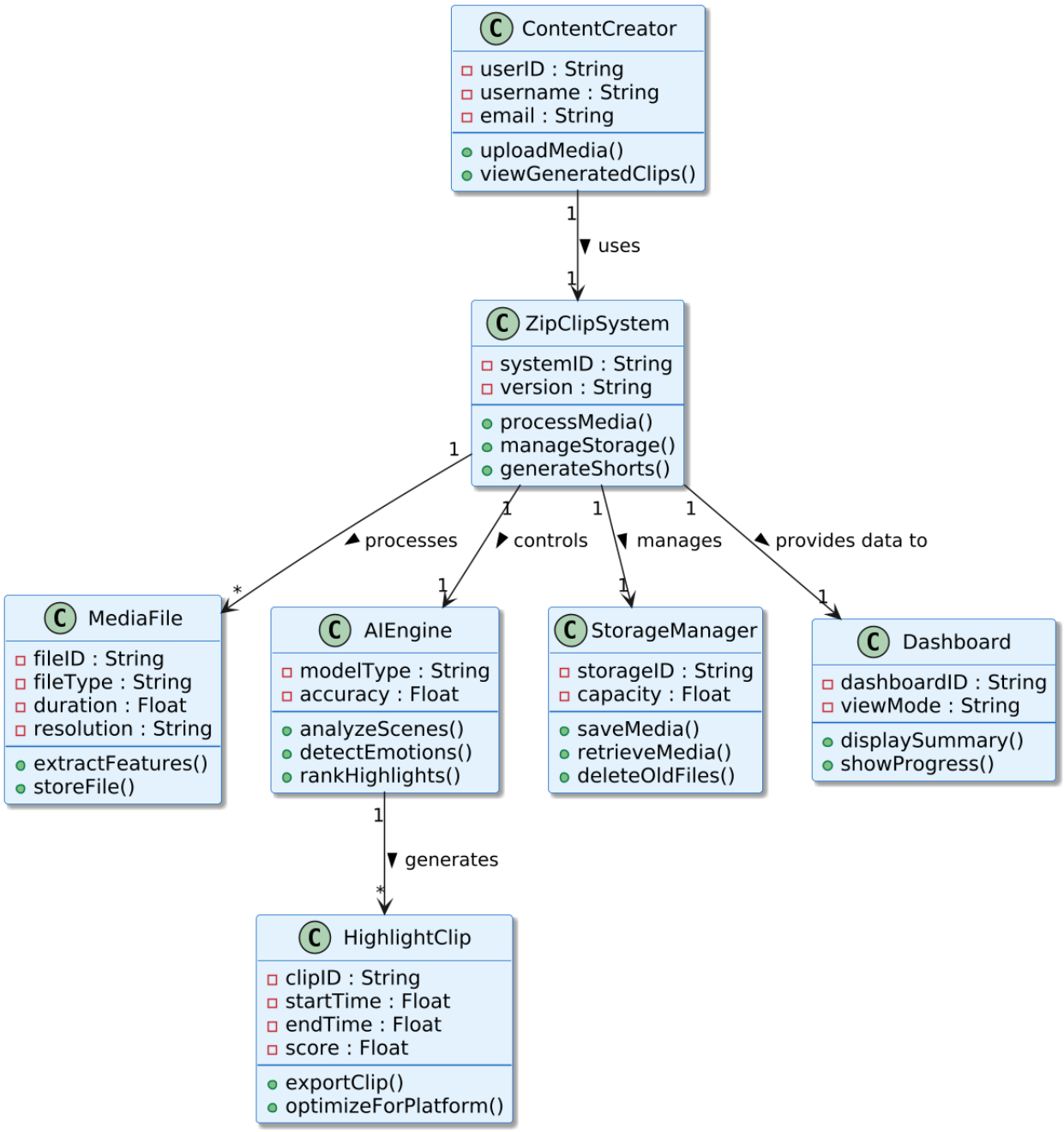

Figure 3.6: Class diagram

3.2 System Requirements

3.2.1 Hardware Requirements

The hardware requirements consist of a computer or laptop with a quad-core processor, 8GB RAM or more, and at least 500GB storage to run the video processing and AI modules. For faster processing, a GPU with CUDA support is recommended, as it accelerates transcription using the Whisper model. A stable internet connection is also required for accessing external APIs.

3.2.2 Software Requirements

The software tools used in this system are mainly for AI processing, video manipulation, and web interface development. The software platforms used in this system are Python, React.js, FFmpeg, Whisper, OpenAI API, and ImageMagick.

3.2.3 Python

Python is a widely used programming language known for its simplicity, readability, and extensive support for artificial intelligence, automation, and multimedia processing. It provides a large ecosystem of libraries and frameworks that make it suitable for building complex applications involving machine learning, data processing, and video analysis. Due to its clear syntax and strong community support, Python has become one of the most popular programming languages for developing AI-based systems and backend services.

Python offers numerous libraries that simplify the development of intelligent systems and multimedia applications. These libraries enable developers to perform tasks such as data manipulation, speech recognition, and video processing efficiently. In the context of artificial intelligence, Python is commonly used for implementing machine learning algorithms, integrating AI models, and managing complex data workflows. Its flexibility also allows seamless integration with external APIs and multimedia tools, making it highly suitable for building modern intelligent applications.

In the ZipClip system, Python is used as the main backend programming language responsible for handling the core processing logic of the application. It manages the video analysis pipeline, integrates AI models such as Whisper for speech transcription, and communicates with the OpenAI API to analyze transcripts and identify highlight segments. Python also coordinates with multimedia processing tools like FFmpeg for video editing tasks such as extracting audio, clipping video segments, and preparing the final output. By acting as the central processing layer of the system, Python enables efficient coordination between artificial intelligence models, video processing tools, and the web interface, ensuring the smooth functioning of the ZipClip platform.

3.2.4 React.js

React.js is a widely used open-source JavaScript library developed by Facebook for building dynamic and interactive user interfaces for modern web applications. It enables developers to create responsive and efficient user interfaces by breaking the UI into reusable components. These components can be independently managed and reused across different parts of an application, which simplifies development and improves maintainability. React.js also uses a virtual DOM (Document Object Model) to efficiently update and render components, resulting in faster performance and smoother user experiences.

One of the key advantages of React.js is its ability to manage application state effectively. Through mechanisms such as state management and component-based architecture, developers can build scalable applications that handle user interactions and data updates efficiently. React also supports integration with various libraries and frameworks, making it suitable for building complex front-end systems that interact with backend APIs and services.

In the ZipClip system, React.js is used to develop the frontend interface that allows users to interact with the application in an intuitive and user-friendly manner. The interface enables users to upload videos or provide YouTube links, initiate the processing workflow, and view the generated short video clips. Through this interface, users can easily manage their video processing tasks and access the final outputs. By providing a clean and responsive dashboard, React.js ensures that users can interact with the ZipClip system smoothly while the backend processes the video content and generates the final short-form clips.

3.2.5 FFmpeg

FFmpeg is a powerful open-source multimedia framework widely used for processing, editing, and converting video and audio files. It supports a wide range of multimedia formats and provides a comprehensive set of tools for tasks such as video encoding, decoding, streaming, filtering, cropping, and format conversion. Due to its efficiency and flexibility, FFmpeg has become one of the most widely used tools in multimedia applications, video editing software, and media streaming platforms.

One of the major advantages of FFmpeg is its ability to handle complex multimedia processing tasks through a wide variety of command-line operations and libraries. It allows developers to manipulate video and audio streams efficiently while maintaining high performance and compatibility with numerous file formats. FFmpeg can also perform operations such as frame extraction, video resizing, bitrate adjustment, and synchronization of audio and video streams. These capabilities make it an essential tool for applications that require automated multimedia processing.

In the ZipClip system, FFmpeg plays a crucial role in handling the video processing pipeline. It is used to extract the audio track from the input video so that the audio can be analyzed by the speech recognition system. Once the relevant segments of the video are identified by the AI model, FFmpeg is used to precisely cut the required video portions from the original video file. The extracted video segments are then transformed into a vertical 16:9 format, which is optimized for short-form video platforms such as YouTube Shorts, Instagram Reels, and other mobile-first media platforms.

Additionally, FFmpeg is responsible for combining the processed video segments with the generated subtitles and audio streams to produce the final output clip. This process ensures that the resulting video maintains synchronization between audio, subtitles, and visual content. By providing efficient multimedia processing capabilities, FFmpeg enables the ZipClip system to automate the transformation of long videos into short, engaging clips suitable for modern digital platforms.

3.2.6 Whisper

Whisper is an advanced automatic speech recognition (ASR) system developed by OpenAI for converting spoken language into written text. It is designed to handle a wide range of speech patterns, accents, and audio conditions, making it a highly reliable tool for transcription tasks. Whisper is trained on a large and diverse dataset of multilingual audio, which allows it to accurately recognize and transcribe speech from different languages and speaking styles. Due to its high accuracy and robustness, Whisper has become a widely used tool in applications involving speech recognition, transcription services, and audio analysis.

One of the key strengths of Whisper is its ability to process audio recordings with background noise, varied speech speeds, and multiple speakers while still maintaining accurate transcription results. The model can automatically detect language, perform speech-to-text conversion, and generate detailed transcripts that represent the spoken content of the audio. This capability makes it particularly useful for applications that require analysis of spoken information within multimedia content.

In the ZipClip system, Whisper plays an important role in the video processing pipeline. After a video is provided as input, the audio track is first extracted from the video using multimedia processing tools. The extracted audio is then passed to the Whisper model, which transcribes the spoken dialogue into text. This transcript acts as a structured representation of the video's spoken content and serves as the basis for further analysis.

The generated transcript is then used by the system to identify meaningful or engaging segments within the video. By analyzing the textual content produced by Whisper, the ZipClip system can

determine which portions of the video contain important information or interesting discussions. These segments are then selected for conversion into short clips suitable for platforms such as YouTube Shorts or Instagram Reels. Through this process, Whisper enables the system to understand and analyze spoken content within videos, making it a crucial component in the automated highlight generation workflow.

3.2.7 OpenAI API

The OpenAI API provides access to advanced artificial intelligence models that are capable of understanding, processing, and analyzing natural language. These models are built using state-of-the-art machine learning techniques and are trained on large datasets to perform tasks such as text generation, summarization, language translation, and contextual analysis. By providing developers with easy access to these powerful models through a web-based API, OpenAI enables the integration of intelligent language processing capabilities into modern applications.

One of the key capabilities of the OpenAI API is its ability to analyze textual data and extract meaningful insights from it. The models can understand context, identify important information, and generate responses or summaries based on the given input. This makes the API highly useful for applications involving natural language processing (NLP), conversational systems, automated content generation, and intelligent data analysis. Through simple API requests, applications can send textual data to the model and receive processed results that help automate complex analytical tasks.

In the ZipClip system, the OpenAI API plays an important role in identifying highlight segments within long videos. After the audio from a video is transcribed into text using the Whisper speech recognition model, the resulting transcript is sent to the OpenAI API for further analysis. The system specifically utilizes the GPT-4o-mini model, which is capable of understanding the context and meaning of the transcript. By analyzing the spoken content of the video, the model identifies the most engaging, informative, or meaningful portions of the transcript.

Based on this analysis, the system determines the time intervals within the video that correspond to these important segments. These selected segments are then used by the system to extract the relevant video portions and convert them into short clips suitable for platforms such as YouTube Shorts or Instagram Reels. Through this intelligent analysis process, the OpenAI API enables ZipClip to automatically detect highlight moments within longer videos, reducing the need for manual editing and making the clip generation process more efficient and scalable.

3.2.8 ImageMagick

ImageMagick is an open-source software suite widely used for image processing and graphic manipulation. It provides a comprehensive set of tools for creating, editing, converting, and displaying images in a variety of formats. The software supports a wide range of image processing operations, including resizing, cropping, color adjustments, filtering, and text rendering. Due to its flexibility and powerful command-line capabilities, ImageMagick is commonly used in applications that require automated image generation and graphic processing.

One of the key features of ImageMagick is its ability to render and manipulate text overlays on images or video frames. This capability allows developers to dynamically generate text-based graphics such as captions, watermarks, labels, and subtitles. The tool also supports customization options for fonts, colors, sizes, positioning, and styling, enabling developers to design visually appealing text elements that enhance the overall presentation of multimedia content.

In the ZipClip system, ImageMagick is used to generate styled subtitles that are embedded into the final short video clips. After the audio from the input video is transcribed using the Whisper speech recognition model, the resulting text is processed to create subtitles that correspond to the spoken dialogue. ImageMagick is then used to render these subtitles with appropriate formatting, including font style, color, outline, and placement within the video frame.

These styled subtitles are then combined with the processed video segments during the final rendering stage. By embedding subtitles directly into the video, the system improves accessibility and viewer engagement, especially for audiences who watch videos without sound or in noisy environments. Through this process, ImageMagick plays an important role in enhancing the visual quality and usability of the short clips generated by the ZipClip system.

3.3 Implementation Details

Implementing **ZipClip: Edit Less, Post More** requires careful coordination of artificial intelligence models, video processing tools, and web technologies.

Firstly, for video processing and analysis, the system accepts either YouTube URLs or locally uploaded video files as input. This flexible input mechanism allows users to process video content from a variety of sources, making the system suitable for both online content creators and users who work with locally stored media. Users who wish to extract highlights from publicly available content can simply provide a YouTube link, while those working with recorded or downloaded media can upload video files directly into the system. This flexibility ensures that the ZipClip system can accommodate a wide range of use cases and user requirements.

Once the video is provided to the system, it is processed using FFmpeg, a powerful multimedia processing framework that handles tasks such as decoding the video stream and preparing the media for further analysis. FFmpeg is responsible for managing the multimedia data contained within the video file and enabling efficient manipulation of video and audio streams. One of the initial steps performed by FFmpeg is the extraction of the audio track from the video file. Separating the audio component allows the system to focus specifically on analyzing the spoken content within the video without interference from the visual data. This step is essential because much of the important contextual information in many videos is conveyed through speech.

The extracted audio is then passed to Whisper, an automatic speech recognition system that converts spoken language into written text. Whisper analyzes the audio waveform and generates a detailed transcript of the spoken dialogue within the video. This transcription process allows the system to convert unstructured audio data into a structured textual format that can be analyzed more effectively by artificial intelligence models. The resulting transcript includes the words spoken in the video along with their sequence and context. This transcript serves as the foundation for identifying meaningful or engaging segments within the video, as it provides a structured representation of the content being discussed.

Secondly, artificial intelligence is used to determine the most engaging portions of the video based on the generated transcript. The textual data produced by Whisper is analyzed using the OpenAI GPT-4o-mini model, which is capable of understanding context, identifying important statements, and recognizing moments of high informational or emotional value within the transcript. By analyzing the semantic meaning of the conversation and evaluating how different parts of the dialogue relate to one another, the AI model can identify which segments are likely to be most interesting or valuable to viewers.

By evaluating the language used in the video, the model can determine which segments are most suitable for conversion into short-form video content. This process enables the system to automatically identify highlight segments without requiring manual editing or human review of the entire video. The AI model considers factors such as clarity of information, significance of statements, and conversational emphasis when determining which segments should be selected. Based on this analysis, the system determines the most relevant time intervals within the video that correspond to these key moments and marks them for clip extraction.

Thirdly, once the highlight segments are identified, FFmpeg is again used to extract the selected portions from the original video file. The system precisely cuts the video according to the time intervals identified during the analysis stage. Accurate trimming ensures that the extracted segment retains the complete context of the highlight without including unnecessary portions of the original

video. These extracted segments are then converted into a vertical video format (9:16) that is optimized for modern short-form video platforms such as YouTube Shorts, Instagram Reels, and other mobile-first media platforms.

During this stage, the system may also perform intelligent cropping techniques that focus on faces, motion areas, or other important visual elements within the frame. Since the original video may have been recorded in a horizontal format, cropping ensures that the most relevant visual information remains visible after the video is converted to a vertical layout. This ensures that viewers can clearly see important visual content even when the video is displayed on mobile devices.

Fourthly, subtitles are automatically generated to improve both accessibility and viewer engagement. The transcription produced by Whisper is used to create subtitle text that corresponds to the spoken dialogue within the selected video segment. These subtitles allow viewers to follow the conversation more easily and improve the overall accessibility of the video, especially for users watching without audio.

The subtitles are then processed using ImageMagick, which renders the text onto the video frames with appropriate styling such as font selection, text color, outline, and positioning. Properly styled subtitles enhance readability and ensure that viewers can follow the content even when watching without sound. The subtitles are synchronized with the audio so that each line appears at the correct moment in the video. This synchronization ensures that the subtitles accurately reflect the spoken dialogue and provide a smooth viewing experience. The subtitles are then embedded directly into the final rendered clip.

Finally, the system integrates a web-based interface developed using React.js, which allows users to interact with the platform in an intuitive and user-friendly manner. Through this interface, users can upload videos, submit YouTube links, initiate the video processing workflow, and preview or download the generated short clips. The frontend interface provides a convenient way for users to interact with the processing system without needing to understand the underlying technical operations.

The frontend interface communicates with backend processing modules that are developed using Python, which manages the core logic of the system. These backend modules coordinate the different stages of the processing pipeline and ensure that each step is executed in the correct sequence. The backend handles tasks such as AI-based transcript analysis, video processing operations, subtitle generation, and communication with external services such as the OpenAI API.

By coordinating these components, the system creates a streamlined workflow that enables the automated generation of short, engaging video clips from longer video content. This integrated

pipeline ensures that each stage of processing—from video input to final clip generation—works together efficiently. As a result, the ZipClip system significantly reduces the time and effort required for manual video editing while enabling users to quickly generate short-form video content that is ready to be shared on modern digital platforms.

3.4 Methodology

3.4.1 Data Collection and Storage

Data collection and storage in the ZipClip system focus on efficient video processing, organized data management, and secure handling of input and output files generated during the processing workflow. Since the system is designed to analyze and process video content, it primarily deals with multimedia data provided by users. Managing multimedia data efficiently is essential because video files are typically large and require careful handling during processing. The system is therefore designed to manage video inputs and generated data in a structured manner to ensure smooth operation and minimal resource usage.

The system accepts video input in two main forms: YouTube URLs and locally uploaded video files. This flexible input mechanism allows users to process videos from online platforms as well as from their personal storage devices. Users who wish to generate short clips from publicly available content can simply provide a YouTube link, while users working with personal or recorded content can upload video files directly into the system. This approach makes the system versatile and adaptable to a variety of user needs and content sources.

Once a video is provided, the system retrieves the video content and prepares it for further analysis and processing. In the case of a YouTube URL, the system downloads the required video stream and converts it into a format suitable for internal processing. For locally uploaded files, the system verifies the video format and prepares it for the processing pipeline. At this stage, the video file becomes the primary source of data used by the subsequent modules of the system.

The collected video data is used only for the purpose of performing automated analysis and generating short video clips. The system extracts relevant information from the video, including audio content and visual segments, which are required for speech transcription and highlight detection. Audio extraction is an important step because it allows the speech recognition system to convert spoken dialogue into text. This textual representation of the spoken content enables the system to perform deeper analysis using artificial intelligence models.

During the processing pipeline, several intermediate data files are created, including extracted audio files, textual transcripts, and temporary video segments. These intermediate files play an

important role in enabling the different stages of the system, such as speech recognition, AI-based content analysis, and clip generation. For example, the extracted audio file is used as input for the Whisper speech recognition model, while the generated transcript is analyzed by the AI system to identify meaningful portions of the video. Temporary video segments may also be created when the system identifies potential highlight intervals that need to be evaluated or processed further.

To ensure efficient system performance and prevent unnecessary storage usage, the ZipClip system handles these intermediate files as temporary data. Since multimedia processing can generate large numbers of intermediate files, storing them permanently would quickly consume storage resources and reduce system efficiency. For this reason, the system carefully manages temporary files and removes them once they are no longer required.

The extracted audio files are generated only for the duration required for speech transcription, while the generated transcripts are used for identifying key moments within the video. Similarly, intermediate video segments may be created during the clip extraction and formatting stages. After the final short video clip is successfully generated, these temporary files are automatically removed from the system to maintain storage efficiency and prevent accumulation of unnecessary data.

The final output of the system is a processed short video clip that contains the selected highlight segment along with additional enhancements such as subtitles and vertical formatting. These output files are stored separately from the temporary processing data to ensure easy access for users. The final clips are typically generated in a format optimized for modern short-form video platforms, allowing users to directly share or upload them to social media platforms.

Proper data management is essential for maintaining the reliability and scalability of the system. By processing only the necessary data and removing temporary files after use, the system ensures that storage resources are used efficiently while maintaining smooth operation of the video processing pipeline. This structured approach to data collection and storage enables the ZipClip system to handle multiple video processing tasks simultaneously while maintaining consistent performance and system stability.

1. Collection of Video Data: The system collects input in the form of a YouTube video link or locally uploaded video file. The uploaded video is used only for generating short clips. Audio and video data required for transcription and analysis are extracted from the original video.

2.Storage of Processing Data: During processing, temporary files such as audio files, transcripts, and intermediate video segments are generated. These files are stored temporarily on the processing server and are used by various modules of the system during the analysis and clip generation stages. Once the processing workflow is completed and the final output video is produced, these temporary files are automatically deleted to ensure efficient use of storage resources.

3.Handling of Output Data: The final processed video clips are saved as output files in vertical short-video format suitable for platforms such as YouTube Shorts and Instagram Reels. These clips contain the selected highlight segments along with embedded subtitles that enhance accessibility and viewer engagement. The output files are stored in a structured format so that users can easily access, download, or share the generated clips.

3.4.2 AI-Based Video Processing

Artificial intelligence forms the core methodology of the ZipClip system, enabling the automated identification and generation of engaging short video clips from longer video content. In traditional video editing workflows, creators often need to manually watch the entire video, identify key moments, trim segments, and then reformat the content to suit different social media platforms. This process is time-consuming and requires both technical expertise and creative judgment. By incorporating artificial intelligence techniques, the ZipClip system automates these processes and significantly reduces the time and effort required for content creation. The integration of AI technologies allows the system to analyze video content intelligently and determine which portions of the video are most suitable for conversion into short-form media. As a result, the system helps users quickly transform long videos into concise and engaging clips that can be easily shared across various digital platforms.

The system combines several advanced technologies, including speech recognition, natural language processing, and multimedia video processing, to achieve this functionality. Each of these technologies plays a specific role in the overall processing pipeline of the system. Speech recognition techniques are used to convert the spoken audio present in the video into textual transcripts. These transcripts provide a structured representation of the dialogue and information contained within the video, making it easier for the system to analyze the content programmatically. Once the transcript is generated, natural language analysis techniques are applied to interpret the meaning, context, and relevance of the spoken content. By understanding the semantics of the conversation, the system can determine which segments contain important information, key explanations, or engaging statements that are more likely to attract viewer attention.

In addition to speech and language analysis, the system also incorporates video processing techniques to handle the multimedia aspects of the content. These techniques enable the extraction, trimming, and formatting of video segments based on the results produced by the AI analysis stage. Video processing tools are responsible for cutting the original video into smaller segments and preparing them in a format that is optimized for modern digital platforms. Once the most relevant sections of the video are identified through AI analysis, the system processes these segments and prepares them for presentation as short clips that are visually appealing and easy to consume.

By combining these technologies, the ZipClip system creates a coordinated workflow that transforms long videos into concise and engaging short clips. This integrated workflow ensures that each stage of processing—from speech transcription to final video generation—works together efficiently to produce high-quality results. This approach not only automates the editing process but also enhances the overall efficiency of content production. As a result, users can quickly generate high-quality short-form videos that are suitable for platforms such as YouTube Shorts, Instagram Reels, and other social media channels, making the system highly useful for content creators, educators, and marketers who frequently produce digital media content.

1.Speech Transcription: The audio extracted from the input video is processed using the Whisper speech recognition model, which converts spoken dialogue into text with high accuracy. This process allows the system to transform the spoken content of the video into a structured textual format that can be easily analyzed by artificial intelligence models. The generated transcript represents the complete spoken content of the video and serves as the foundation for further analysis. By converting audio into text, the system gains the ability to understand and analyze the informational context of the video more effectively.

2.Highlight Detection: The generated transcript is analyzed using the OpenAI GPT-4o-mini model to identify meaningful and engaging segments within the video. The AI model evaluates the context of the conversation, identifies key ideas, and determines which portions of the dialogue are most relevant or interesting to viewers. Based on this analysis, the system selects the most appropriate time intervals within the video that should be converted into short clips. This automated highlight detection process significantly reduces the need for manual review of the entire video.

3.Clip Extraction: Once the relevant segment is identified, FFmpeg is used to extract the selected time interval from the original video file. The system carefully trims the video according to the start and end times determined by the AI model. The extracted clip represents the key highlight identified during the analysis stage and serves as the primary content for the short-form video.

4.Video Formatting: The extracted clip is converted into a vertical video format (9:16) suitable for platforms such as YouTube Shorts and Instagram Reels. Since many modern social media platforms prioritize mobile viewing, vertical video formats provide a better viewing experience on smartphones. Intelligent cropping techniques are applied to ensure that important visual elements such as faces, motion areas, or key objects remain centered within the frame. This helps maintain visual clarity even after the video is reformatted.

5.Subtitle Generation: To improve accessibility and viewer engagement, subtitles are generated from the transcript and embedded directly into the video. These subtitles are rendered using ImageMagick, which allows customization of font style, text color, outline, and positioning. The subtitles are synchronized with the spoken content so that viewers can easily follow the dialogue while watching the video. This feature is particularly useful for viewers who watch videos without sound or in noisy environments.

Chapter 4

CONCLUSION

In conclusion, the proposed ZipClip: AI-Powered Video Highlight Generator has the potential to significantly simplify the process of creating engaging short-form video content from longer videos. Through the integration of artificial intelligence, speech recognition, and advanced video processing technologies, this system aims to address the challenges faced by content creators in manually editing and identifying highlight segments within long videos. By utilizing tools such as Whisper for speech transcription, OpenAI models for intelligent highlight detection, and FFmpeg for efficient video processing, the system automates the transformation of long-form videos into short, engaging clips suitable for modern social media platforms. Furthermore, ZipClip provides users with an efficient platform to generate vertical short videos with automatically generated subtitles, improving accessibility and viewer engagement. The integration of a web-based interface allows users to easily upload videos, process content, and obtain ready-to-share short clips without requiring advanced video editing skills. Additionally, the system supports multiple processing modes that enable flexible handling of different types of video content, making it suitable for creators, educators, and marketers. Overall, ZipClip offers an innovative solution for automated video highlight generation, helping users quickly convert long videos into engaging short clips while reducing manual editing effort. This approach enhances productivity for content creators and supports the growing demand for short-form video content across digital platforms.

References

[1] H. Gu, S. Petrangeli and V. Swaminathan, ”SumBot: Summarize Videos Like a Human,” 2020 IEEE International Symposium on Multimedia (ISM), Naples, Italy, 2020, pp. 210-217.

[2] X. Zhou, J. Smith, “Video summarization using deep learning techniques: a comprehensive review,” ACM Computing Surveys, vol. 56, no. 3, pp. 44:1–44:39, 2023.

[3] Y. Chen, L. Patel, “Video summarization for event-centric videos,” IEEE TNNLS, vol. 34, no. 7, pp. 1126–1138, 2023.

[4] M. Gupta, et al, “AI-driven video summarization for optimizing content search and retrieval efficiency,” Sci Rep, vol. 15, no. 2, pp. 141–150, 2025.

[5] Q. Li, A. Tang, “Video summarization via knowledge-aware multimodal networks,” Inf Fus, vol. 99, pp. 191–200, 2024.

[6] Y. Wang, J. Lee, “Zero-shot scene change detection,” in AAAI Conf Artif Intell, ACM, vol. 39, no. 3, pp. 32253–32260, 2024.

[7] W. Fan, C. Liu, “Scene Object Change Detection and Fine-Grained Target Recognition,”

[8] C. Chang, Z. Liu, “Advances in Video Emotion Recognition: Challenges and Progress,” IEEE Trans Affect Comput, vol. 16, no. 4, pp. 360–373, 2025.

[9] F. Silva, R. Rao, “Multimodal Emotion Recognition with Deep Learning,” Inf Fusion, vol. 79, pp. 33–44, 2024.

[10] D. Wu, S. Zhou, “Applications and Reliability of Automatic Facial Emotion Recognition in Video Analysis,” ACM TMM, vol. 24, no. 5, pp. 452–463, 2024.

[11] . Park, S. Kim, “Emotion Recognition using multi-modal data and self-supervised learning,” ACM TMM, vol. 31, no. 2, pp. 339–347, 2024.

[12] S. Agarwal, H. Singh, “Real Time Emotion Recognition Using Image Classification,” ACM, ICIP, pp. 276–285, 2023.

Appendix A

Backend/Processing Module

```
# Import required libraries for video processing, numerical operations,
and system utilities

from moviepy.editor import VideoFileClip, concatenate_videoclips, CompositeVideoClip,
ColorClip, vfx, AudioFileClip, CompositeAudioClip
import subprocess  # Used for running system-level commands if needed
import numpy as np  # Used for numerical operations (arrays, calculations)
import os  # Used for file handling and path operations
import random  # Used for random selection (transitions, assets)
import cv2  # OpenCV used for image/frame processing


# Function to extract audio from a video file
def extractAudio(video_path, audio_path="audio.wav"):
    try:
        # Load video file
        video_clip = VideoFileClip(video_path)

        # Extract and save audio
        video_clip.audio.write_audiofile(audio_path)

        # Close video to release memory
        video_clip.close()

        print(f"Extracted audio to: {audio_path}")
```

```
        return audio_path

    except Exception as e:
        # Handle errors during extraction
        print(f"An error occurred while extracting audio: {e}")
        return None


# Function to crop a specific segment from a video
def crop_video(input_file, output_file, start_time, end_time):
    with VideoFileClip(input_file) as video:

        # Ensure end_time doesn't exceed video duration
        max_time = video.duration - 0.1  # Small buffer to avoid edge cases
        if end_time > max_time:
            print(f"Warning: Requested end time ({end_time}s) exceeds video duration
            ({video.duration}s). Capping to {max_time}s")
            end_time = max_time

        # Extract subclip
        cropped_video = video.subclip(start_time, end_time)

        # Save cropped segment
        cropped_video.write_videofile(output_file, codec='libx264')


# Function to stitch multiple video segments together with intelligent transitions
def stitch_video_segments(input_file, segments, output_file, theme=None):
    """
    Extract multiple segments from a video and stitch them together.

    Args:
        input_file: Path to the input video file
        segments: List of dicts with 'start' and 'end' keys
        output_file: Path for output video
        theme: Optional theme to influence transitions
```

```
    Returns:
        True if successful, False otherwise
    """
    try:
        print(f"\nStitching {len(segments)} video segments...")

        # Load main video file
        video = None
        if input_file:
            video = VideoFileClip(input_file)

        # Cache video handles to avoid reopening files repeatedly
        video_cache = {}
        if input_file:
            video_cache[input_file] = video

        # Helper function to fetch video from cache
        def get_video_from_cache(path):
            if path not in video_cache:
                print(f"  Opening new file handle for: {os.path.basename(path)}")
                video_cache[path] = VideoFileClip(path)
            return video_cache[path]

        # Initialize storage for clips
        clips = []
        total_duration = 0
        target_size = None

        # Loop through all segments
        for i, segment in enumerate(segments):

            # Check if direct clip object or file path is provided
            clip_obj = segment.get('clip')
            seg_file = segment.get('file_path', input_file)
```

```
if clip_obj:
    print(f"  Using direct clip object for segment {i+1}")
    temp_video = clip_obj
elif seg_file and os.path.exists(seg_file):
    try:
        temp_video = get_video_from_cache(seg_file)
    except Exception as e:
        print(f"  Error opening {seg_file}: {e}")
        continue
else:
    print(f"  Warning: No source found for segment {i+1}")
    continue

# Prevent exceeding duration
seg_max_time = temp_video.duration - 0.1
start = segment['start']
end = segment['end']

# Validate segment timing
if end > seg_max_time:
    print(f"  Warning: Segment {i+1} end time exceeds duration")
    end = seg_max_time

if start >= end:
    print(f"  Warning: Skipping invalid segment {i+1}")
    continue

# Extract segment
print(f"  Extracting segment {i+1}")
clip = temp_video.subclip(start, end)

# Ensure consistent resolution across clips
if target_size is None:
    tw, th = temp_video.size if hasattr(temp_video, 'size')
    else clip.size
    if tw % 2 != 0: tw -= 1
```

```
            if th % 2 != 0: th -= 1
            target_size = (tw, th)

        cw, ch = clip.size
        tw, th = target_size

        # Resize if dimensions mismatch
        if cw != tw or ch != th:
            from moviepy.video.fx.all import resize, crop
            if cw/ch > tw/th:
                clip = resize(clip, height=th)
            else:
                clip = resize(clip, width=tw)
            clip = crop(clip, x_center=clip.w/2,
            y_center=clip.h/2, width=tw, height=th)

        clips.append(clip)
        total_duration += (end - start)

    # Check if clips exist
    if not clips:
        print("Error: No valid clips to stitch")
        for v in video_cache.values():
            v.close()
        return False

    print(f"Total duration: {total_duration:.2f}s")

    # Helper function: safely extract frame
    def _get_frame_safe(clip, t_offset=0.1):
        try:
            t = min(max(t_offset, 0), max(0, clip.duration - 0.01))
            return clip.get_frame(t)
        except Exception:
            return None
```

```
# Helper function: calculate frame difference
def _frame_diff(f1, f2):
    if f1 is None or f2 is None:
        return 1.0
    try:
        f1 = f1.astype('float32') / 255.0
        f2 = f2.astype('float32') / 255.0
        return float(np.mean(np.abs(f1 - f2)))
    except Exception:
        return 1.0

# Detect if theme is celebratory
is_celebration = theme and any(kw in theme.lower()
for kw in ['birthday', 'party', 'celebration', 'festive'])

# Transition selection logic
def _choose_transition(clip_a, clip_b):

    # Short clips → cut
    if clip_a.duration < 1.5 or clip_b.duration < 1.5:
        return ('cut', 0)

    # Compare frames for similarity
    frame_a = clip_a.get_frame(max(0, clip_a.duration - 0.05))
    frame_b = clip_b.get_frame(0.05)
    diff = _frame_diff(frame_a, frame_b)

    # Choose transition based on similarity
    if diff < 0.15:
        return ('crossfade', 1.0)
    if diff < 0.30:
        return ('fade', 1.0)

    return ('light_leak', 0.8)

# Timeline construction
```

```
timeline_clips = []
overlays = []
current_time = 0.0

for i, clip in enumerate(clips):

    if i == 0:
        timeline_clips.append(clip.set_start(0))
        current_time += clip.duration
        continue

    prev = clips[i-1]
    trans_type, trans_dur = _choose_transition(prev, clip)

    # Apply transitions
    if trans_type == 'cut':
        timeline_clips.append(clip.set_start(current_time))
        current_time += clip.duration

    elif trans_type == 'fade':
        clip = clip.fx(vfx.fadein, trans_dur)
        timeline_clips.append(clip.set_start(current_time))
        current_time += clip.duration

    elif trans_type == 'crossfade':
        clip_start = current_time - trans_dur
        timeline_clips.append(clip.crossfadein(trans_dur)
        .set_start(clip_start))
        current_time = clip_start + clip.duration

# Create final video composition
final_comp = CompositeVideoClip(timeline_clips)

# Set output resolution
w, h = video.size if video else (timeline_clips[0].w, timeline_clips[0].h)
if w % 2 != 0: w -= 1
```

```
        if h % 2 != 0: h -= 1

        # Export final video
        final_comp.write_videofile(
            output_file,
            codec='libx264',
            audio_codec='aac'
        )

        # Cleanup
        final_comp.close()
        for v in video_cache.values():
            v.close()

        print(" Video stitching successful")
        return True

    except Exception as e:
        print(f"Error stitching video segments: {e}")
        return False


# Function to add background music with ducking effect
def apply_background_music(video_path, music_path, transcriptions, output_path,
                           music_volume=0.3, voice_volume=1.0, ducking_volume=0.08):

    try:
        print(f"Applying background music...")

        video = VideoFileClip(video_path)
        music = AudioFileClip(music_path)

        # Loop music if shorter than video
        if music.duration < video.duration:
            import math
            n_loops = math.ceil(video.duration / music.duration)
```

```
    from moviepy.audio.AudioClip import concatenate_audioclips
    music = concatenate_audioclips([music] * n_loops)

music = music.subclip(0, video.duration)

# Ducking logic (reduce music during speech)
def ducking_filter(get_frame, t):
    t_arr = np.array(t)
    vol = np.ones_like(t_arr, dtype=float)

    for seg in transcriptions:
        mask = (t_arr >= seg['start'] - 0.3) & (t_arr <= seg['end'] + 0.3)
        vol = np.where(mask, ducking_volume, vol)

    frame = get_frame(t)
    return frame * vol

ducked_music = music.fl(ducking_filter).volumex(music_volume)

# Combine original and background audio
if video.audio:
    original_audio = video.audio.volumex(voice_volume)
    final_audio = CompositeAudioClip([original_audio, ducked_music])
else:
    final_audio = ducked_music

final_video = video.set_audio(final_audio)

# Export final video
final_video.write_videofile(
    output_path,
    codec='libx264',
    audio_codec='aac'
)

video.close()
```

```
        music.close()

        print(" Background music applied successfully")
        return True

    except Exception as e:
        print(f"Error applying background music: {e}")
        return False
```

Appendix B

User Interface

```
<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Character encoding and responsive settings -->
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <!-- Page title and SEO description -->
  <title>ZipClip | AI Video Shorts Generator</title>
  <meta name="description" content="Transform long videos into viral YouTube Shorts
  with AI. Upload a video or paste a URL to get started." />

  <!-- External stylesheet -->
  <link rel="stylesheet" href="style.css" />
</head>

<body>

<!-- Main application wrapper -->
<div class="app-wrapper">

  <!-- ================ HEADER SECTION ================ -->
  <header class="app-header">

    <!-- Logo/Icon -->
    <div class="logo-icon"></div>
```

```
  <!-- Title and subtitle -->
  <div class="header-text">
    <h1>ZipClip</h1>
    <p>AI-powered YouTube Shorts generator</p>
  </div>

  <!-- API Health Indicator -->
  <div class="api-health">
    <div id="api-health-indicator" class="health-dot offline"></div>
    <span id="api-health-label">API Offline</span>
  </div>

</header>

<!-- ================ INPUT CARD ================ -->
<div class="card" id="input-card">
  <p class="section-label">Input</p>

  <!-- Tabs for switching between File Upload and URL -->
  <div class="tabs" role="tablist">
    <button class="tab-btn active" id="tab-file" onclick="switchTab('file')">
       Upload File
    </button>
    <button class="tab-btn" id="tab-url" onclick="switchTab('url')">
       Video URL
    </button>
  </div>

  <!-- File Upload Panel -->
  <div class="tab-panel active" id="panel-file">
    <div class="drop-zone" id="drop-zone">

      <!-- File input -->
      <input type="file" id="file-input" accept="video/*" />
```

```
    <!-- UI Elements -->
    <span class="drop-icon"></span>
    <h3>Drop your video here</h3>
    <p>or click to browse · Supported formats up to 500MB</p>

    <!-- Display selected file -->
    <div class="file-selected-name" id="file-name-display"></div>

  </div>
</div>

<!-- URL Input Panel -->
<div class="tab-panel" id="panel-url">
  <div class="input-group">
    <label for="video-url">Video URL</label>
    <input type="url" id="video-url" placeholder="Enter video URL..." />
  </div>
</div>

<!-- ================ OPTIONS ================ -->

<!-- Processing Mode and Duration -->
<div class="options-grid" style="margin-top:22px;">
  <div class="input-group">
    <label for="mode-select">Processing Mode</label>
    <select id="mode-select">
      <option value="continuous">Continuous</option>
      <option value="multi_segment" selected>Multi-Segment</option>
      <option value="scene_based">Scene-Based</option>
    </select>
  </div>

  <div class="input-group">
    <label for="duration-input">Target Duration</label>
    <input type="number" id="duration-input" value="60" />
  </div>
```

```
</div>

<!-- Toggle Options -->
<div style="margin-top:18px;">

  <!-- Subtitle toggle -->
  <div class="toggle-row">
    <div>
      <span>Add Subtitles</span>
      <small>Burn captions into video</small>
    </div>
    <label class="toggle">
      <input type="checkbox" id="toggle-subtitles" checked />
    </label>
  </div>

  <!-- Auto approve toggle -->
  <div class="toggle-row">
    <div>
      <span>Auto-Approve</span>
      <small>Skip manual review</small>
    </div>
    <label class="toggle">
      <input type="checkbox" id="toggle-auto-approve" checked />
    </label>
  </div>

</div>

<!-- Advanced Settings -->
<button onclick="toggleAdvanced()"> Subtitle Styling</button>

<div id="advanced-section">
  <!-- Subtitle styling inputs -->
  <input type="text" id="sub-font" value="Franklin-Gothic" />
  <input type="number" id="sub-fontsize" value="80" />
```

```
    <input type="color" id="sub-color" />
  </div>

  <!-- Submit Button -->
  <button onclick="submitJob()">
     Generate Short
  </button>

</div>

<!-- ================ PROGRESS SECTION ================ -->
<div class="card" id="progress-card">
  <h2>Processing...</h2>

  <!-- Progress bar -->
  <div class="progress-bar-track">
    <div class="progress-bar-fill" id="progress-bar"></div>
  </div>

  <!-- Progress text -->
  <span id="progress-message">Starting...</span>
  <span id="progress-pct">0%</span>
</div>

<!-- ================ RESULT SECTION ================ -->
<div class="card" id="result-card">

  <!-- Status -->
  <h2 id="result-title"></h2>

  <!-- Metadata -->
  <div id="result-meta"></div>

  <!-- Output -->
  <div id="result-body"></div>
```

```
    <!-- Reset button -->
    <button onclick="resetToInput()">Process Another Video</button>

  </div>

  <!-- ================ JOB HISTORY ================ -->
  <div class="card" id="jobs-card">
    <p>Recent Jobs</p>
    <button onclick="loadJobs()">Refresh</button>

    <div id="jobs-list">
      No jobs yet.
    </div>
  </div>

</div>

<!-- Toast Notification Container -->
<div id="toast-container"></div>

<!-- Main JavaScript file -->
<script src="app.js"></script>

</body>
</html>



/* ===== ZipClip Web UI | app.js ===== */

/* Base API URL */
const API_BASE = 'http://localhost:8000';

/* ================ STATE VARIABLES ================ */
let currentJobId = null;   // Stores current job ID
let pollInterval = null;   // Stores polling interval reference
let activeTab = 'file';    // Tracks active input mode (file/url)
```

```
/* ================ TAB SWITCHING ================ */
function switchTab(tab) {
    activeTab = tab;

    // Toggle active styles
    document.getElementById('tab-file').classList.toggle('active', tab === 'file');
    document.getElementById('tab-url').classList.toggle('active', tab === 'url');

    // Show respective panels
    document.getElementById('panel-file').classList.toggle('active', tab === 'file');
    document.getElementById('panel-url').classList.toggle('active', tab === 'url');
}


/* ================ ADVANCED SETTINGS ================ */
function toggleAdvanced() {
    document.getElementById('advanced-section').classList.toggle('open');
}


/* ================ FILE HANDLING ================ */
function setSelectedFile(file) {
    document.getElementById('file-name-display').textContent = file.name;
}


/* ================ GET USER OPTIONS ================ */
function getOptions() {
    return {
        mode: document.getElementById('mode-select').value,
        target_duration: parseInt(document.getElementById('duration-input').value),
        add_subtitles: document.getElementById('toggle-subtitles').checked,
        auto_approve: document.getElementById('toggle-auto-approve').checked
    };
```

```
}


/* ================ SUBMIT JOB ================ */
async function submitJob() {

    const options = getOptions();

    try {
        let jobData;

        // Check active tab
        if (activeTab === 'file') {
            jobData = await uploadFile(options);
        } else {
            jobData = await submitUrl(options);
        }

        // Store job ID
        currentJobId = jobData.job_id;

        // Show progress
        showProgress(jobData);
        startPolling(jobData.job_id);

    } catch (err) {
        console.error(err);
    }
}


/* ================ API CALLS ================ */

/* Upload file */
async function uploadFile(options) {
    return fetch(`${API_BASE}/api/process`, {
```

```
        method: 'POST'
    }).then(res => res.json());
}

/* Submit URL */
async function submitUrl(options) {
    return fetch(`${API_BASE}/api/process`, {
        method: 'POST'
    }).then(res => res.json());
}


/* ================ HEALTH CHECK ================ */
async function checkHealth() {
    try {
        const res = await fetch(`${API_BASE}/health`);

        if (res.ok) {
            document.getElementById('api-health-label').textContent = 'Online';
        }
    } catch {
        document.getElementById('api-health-label').textContent = 'Offline';
    }
}


/* ================ POLLING ================ */
function startPolling(jobId) {
    stopPolling();
    pollInterval = setInterval(() => pollStatus(jobId), 2000);
}

function stopPolling() {
    if (pollInterval) clearInterval(pollInterval);
}
```

```
async function pollStatus(jobId) {
    const res = await fetch(`${API_BASE}/api/status/${jobId}`);
    const job = await res.json();

    updateProgress(job);

    if (job.status === 'completed') {
        stopPolling();
        showResult(job);
    }
}


/* ================ UI HANDLING ================ */

function showProgress(job) {
    document.getElementById('input-card').style.display = 'none';
    document.getElementById('progress-card').style.display = 'block';
}

function updateProgress(job) {
    document.getElementById('progress-bar').style.width = `${job.progress}%`;
}

function showResult(job) {
    document.getElementById('progress-card').style.display = 'none';
    document.getElementById('result-card').style.display = 'block';
}


/* ================ RESET ================ */
function resetToInput() {
    document.getElementById('input-card').style.display = 'block';
    document.getElementById('result-card').style.display = 'none';
}
```

```
/* ================= JOB LIST ================= */
async function loadJobs() {
    const res = await fetch(`${API_BASE}/api/jobs`);
    const jobs = await res.json();

    document.getElementById('jobs-list').innerHTML =
        jobs.map(j => `<div>${j.job_id}</div>`).join('');
}


/* ================= INIT ================= */
function init() {
    checkHealth();
    loadJobs();
}

init();
```

Appendix C

Screenshots

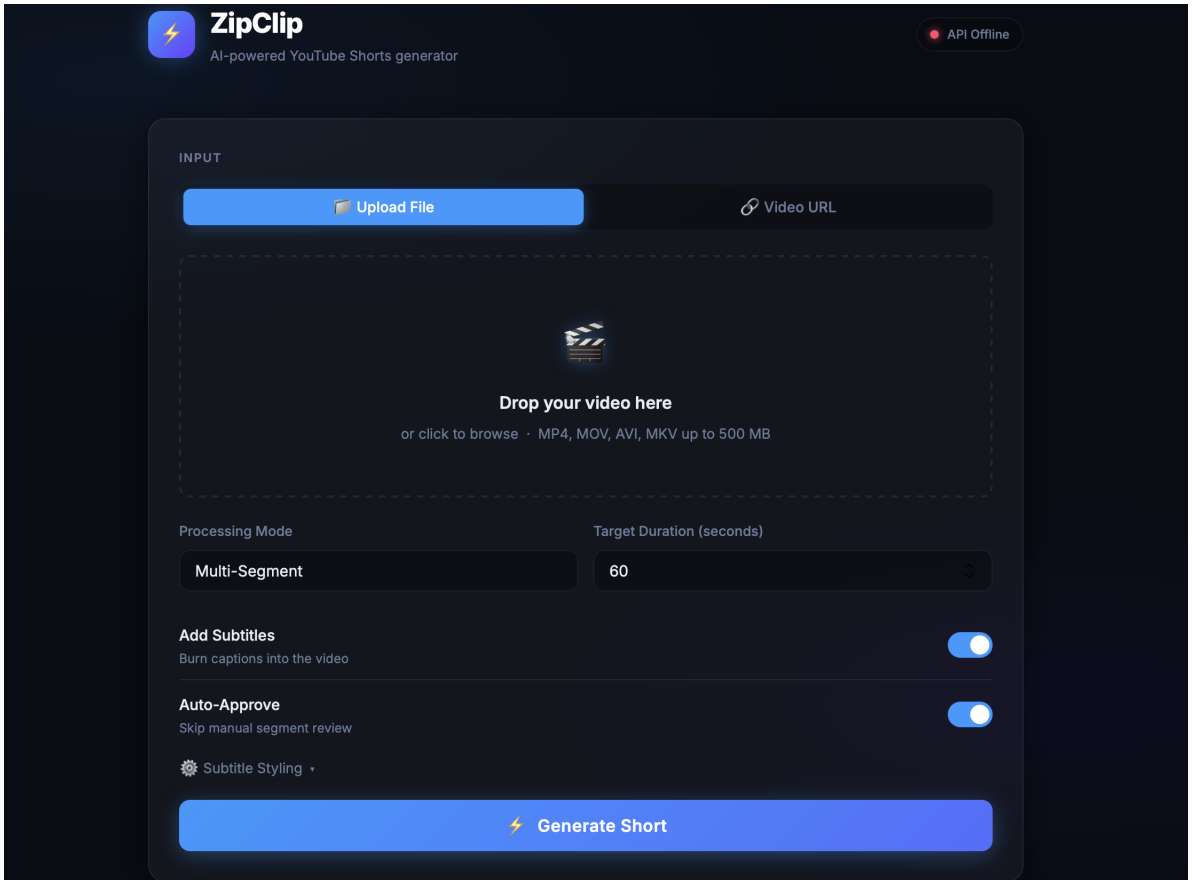

Figure C.1: Home Page-1

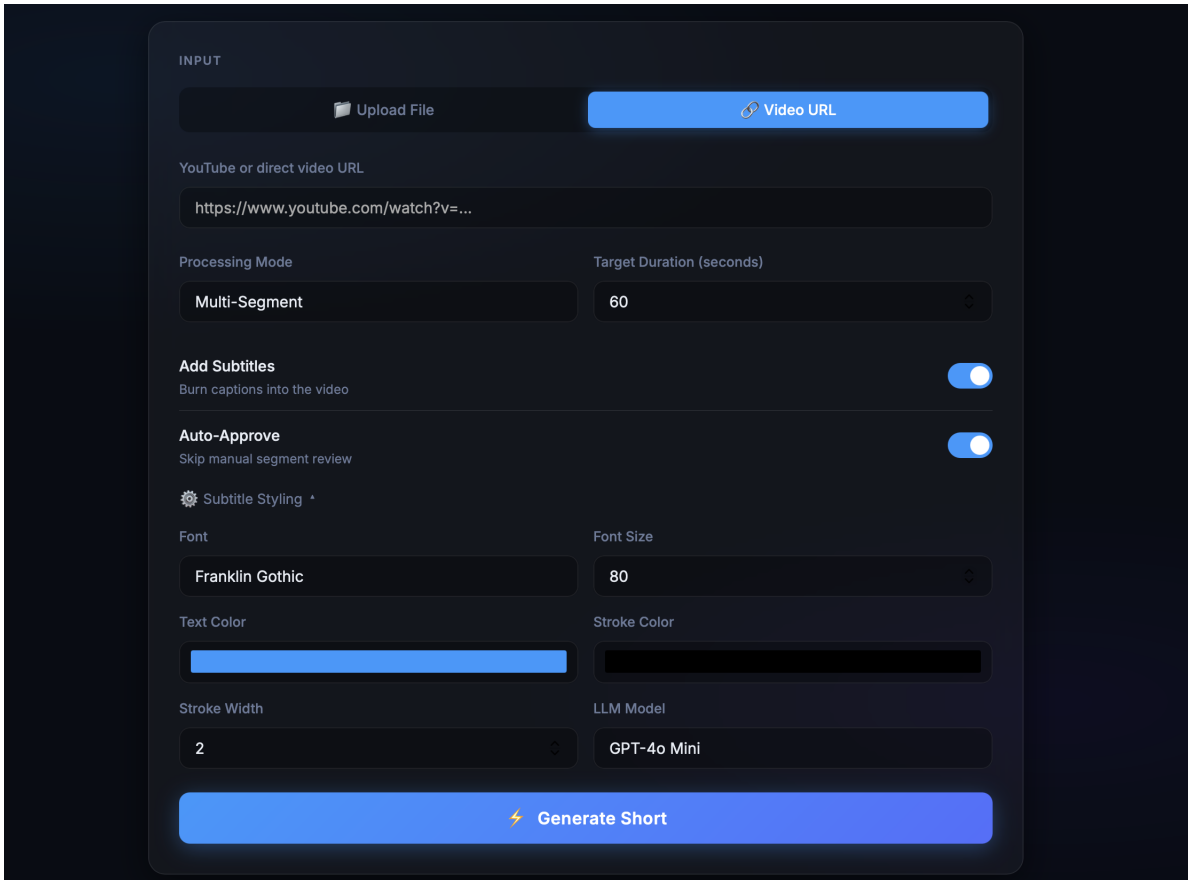

Figure C.2: Home Page-2

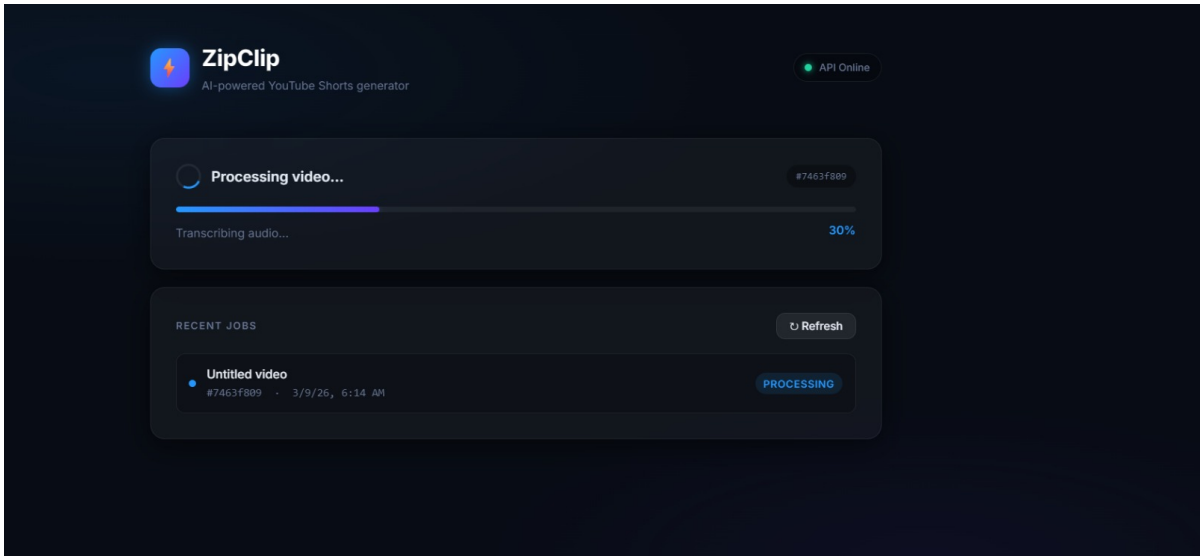

Figure C.3: Processing Page